TrustScientist: Evidence and Branch Revalidation for Autonomous Research Agents
Technical Appendix: Methods, Results and Archived Protocols

Xinyue Fang* Dong Huang Aoqian Zhang Shuzhi Sam Ge

Department of Electrical and Computer Engineering,
National University of Singapore, Singapore

Abstract

Complete technical material accompanying the IET Software revision of 19 October 2026. The first part retains the full method, conditional proofs, study protocols, development rounds, repaired V5 costs, GPT-6 assessments and generated-workflow rounds WF1–WF3 moved from the longer main paper. The selective-replay contract and its analytic witnesses are preserved in full. The second part retains the original archived protocols and figures. Study populations and cost boundaries remain separate; no saved response is relabeled as new inference, and no model judgment is described as human ground truth.

Contents

I Complete Methods and Study Records **4**

1 Evidence and Branch Revalidation **4**
1.1 Requirements, evidence, and closure 4
1.2 Evidence acquisition and certificate revalidation 4
1.3 Decision receipts and the replay boundary 5
1.4 Progress-preserving counterfactual branch revalidation 5
1.5 Conditional consistency, progress, and costs 6
1.6 Scope of closure and verification errors 7

2 Complete Study Results and Development Record **9**
2.1 Evaluation map and recorded examples 9
2.2 Historical branch revalidation 9
2.3 Certificate revalidation 10
2.4 Evidence acquisition 11
2.5 Artifact contract enforcement 13
2.6 Semantic verification 15
2.7 Consecutive updates and bounded replay 16
2.8 Generated-analysis pilot on eight existing research projects 18
2.9 Verified incremental redecision with a matched fine-grained comparator 21

*Corresponding author: xinyue.fang@u.nus.edu.

2.10 Token optimization: failed batching and certified finite dispatch 23
2.11 Repairing an omitted requirement and rerunning matched controls 24

3 Generated Agent workflows: protocol, failures and costs 27
3.1 EXT1: frozen LangGraph integration on new task families 30

4 Exact Recovery Contract and Selective-Replay Boundary 34
4.1 Software contract relative to selective replay 34

II Original Archived Protocols 37

5 Recorded Case Studies 37
5.1 Restoring a reporting state through two evidence actions 37
5.2 A shared source invalidates several certificates 37
5.3 The first affected decision need not be the rollback point 38
5.4 A matching value can still bind to the wrong metric 38

6 Metrics and Repeated Decisions 38
6.1 Rationale-hit diagnostic 39
6.2 The original matched-coverage analysis 39

7 CORE Manifest Protocol and Baselines 40
7.1 Cumulative gates and restoration 41

8 Fresh-Workspace Diagnostic 41

9 Exploratory Natural-Claim Preannotation 43

10 Evidence Archive and Assistance Disclosure 44

11 Archived Restoration Trace 44
11.1 Worked trace: a stale artifact manifest 44

12 Controller Protocol and Historical Evidence 45
12.1 Earlier formal artifact protocol 45
12.2 Additional fresh execution diagnostic 46

13 Operational Labels and Policy Certification 48
13.1 What the existing labels establish 48
13.2 A sufficient episode-level calibration construction 48

14 Paired Semantic-Gate Protocol 50
14.1 Paired design and independent scoring 50
14.2 Execution and uncertainty 50
14.3 Complete paired-gate results 51

15 Dynamic Version-Contract Protocol **53**
15.1 Inputs, checks, and fixed event sequence . . . 53
15.2 Prompted stateful comparator . . . 53
15.3 Denominators, uncertainty, and costs . . . 54
15.4 Executed shared-source failure . . . 54

16 Discriminating Evidence Acquisition **56**
16.1 New acquisition cohort and frozen protocol . . . 56
16.2 Shared state and competing selectors . . . 56
16.3 Scoring and implementation checks . . . 57
16.4 Frozen pilot outcomes and mechanism diagnosis . . . 57
16.5 Post hoc filtered-choice diagnostic . . . 58

17 CBR Implementation and Replay Protocol **60**
17.1 Integration boundary . . . 60
17.2 Executed branch trace . . . 60
17.3 Accounting and limitations . . . 60
17.4 Assistance and artifact . . . 61

18 Preserved Protocol Details **61**
18.1 Selective revalidation of changed inputs . . . 61

19 SciFact: A Fixed-Retrieval Semantic Slice **62**
19.1 Task, controls, and metrics . . . 62
19.2 Utility, errors, and the consensus comparator . . . 62

20 Recorded Result Visualizations **64**
20.1 Reproduction and interpretation . . . 64
20.2 Maintenance and failure modes . . . 64

21 Executed Longer Histories and Continuation **68**
21.1 Histories, updates, and independent scoring . . . 68
21.2 Accounting and matched baseline . . . 68
21.3 An executed rollback and continuation . . . 69
21.4 Original agent-history gap . . . 70

Part I
Complete Methods and Study Records

1 Evidence and Branch Revalidation

TrustScientist maintains two records: a certificate of the current evidence checks and a receipt of each historical decision. The certificate identifies the represented obligations, their artifact bindings, and the checks that passed. The receipt lets CBR reconsider an action when those inputs change. Failed checks remain linked to their originating requirements during acquisition and re-audit.

1.1 Requirements, evidence, and closure

An input consists of a claim c, a represented mandatory requirement set $R(c)$, and an artifact state E. Optional observations are stored outside $R(c)$. Each requirement stores its claim or rubric origin, admissible artifact types, binding rule, and predicate. Artifact records identify content, location, version or digest, and execution metadata. A value match satisfies its numeric predicate; repetitions and baseline comparisons require their own evidence.

The four requirement types organize what must be checked. *Authenticity* (AUTH) checks provenance and agreement between a reported value and its artifact. *Relevance* (REL) checks the claim–artifact relation, including the task, dataset, split, metric, or local question. *Sufficiency* (SUFF) checks specified evidence coverage, such as required replications, uncertainty estimates, or baselines. *Discriminativeness* (DISC) checks whether the evidence separates the claim from a named alternative. The latter two require an operational test supplied by the claim or rubric. The type names alone do not define a universal standard of scientific adequacy.

A requirement is CLOSED when admissible evidence is bound and its predicate passes. It is OPEN when evidence is missing or a check fails and an evidence action can address the gap. It is UNVERIFIABLE when the representation or available verifier cannot reliably decide the predicate. A missing expected file can be OPEN; an uninterpretable semantic relation can be UNVERIFIABLE. Both prevent claim closure. In the CORE protocol, the 17 unbound requirements are recorded as OPEN throughout this paper, matching the experiment's operational treatment.

Every represented mandatory requirement must be CLOSED before the claim closes. Optional findings remain visible without blocking it. The certificate records the claim and requirement-set version, artifact bindings, predicate outcomes, verifier versions, and unresolved reasons. It attests that $R(c)$ passes those checks on the inspected artifact versions. It does not certify that all scientifically necessary requirements were generated or satisfied.

1.2 Evidence acquisition and certificate revalidation

An Evidence-Obligation Graph $G = (V, D)$ connects executable checks for the represented requirements. An edge $u \to v$ means that v depends on the evidence established by u. Each node stores a predicate, artifact bindings, their content digests, a state, and an action specification. A certificate expires when a bound artifact changes. Our reporting implementation uses six nodes: access, integrity, task binding, extraction receipt, literal quotation, and semantic adequacy. Access precedes integrity and binding; both precede extraction, followed by quotation and semantic checks. The six-node graph is an authored, executable reporting schema.

The blocking frontier contains failed or unresolved checks whose prerequisites have all passed. The controller chooses an affordable action with the largest estimated number of repaired checks

per cost unit, then subtracts its cost from the remaining budget. One action can refresh several affected checks. The fixed tool-effect map supplies the gain estimate without reference answers; this ranking is a greedy heuristic. After execution, the controller audits the small graph again and stores the new evidence digest. It suppresses an unchanged unsuccessful action. Budget exhaustion leaves the claim OPEN with the outstanding checks recorded.

Certificate updates. For directly affected checks I_t, the existing tracker invalidates $U_t = I_t \cup \mathrm{Desc}_G(I_t)$ and rechecks them in topological order. With complete dependencies and deterministic predicates, unaffected checks remain unchanged and induction gives full-revalidation equivalence. The appendix supplies the implementation details. Its outputs are refreshed certificates; historical branch actions are maintained by CBR below.

1.3 Decision receipts and the replay boundary

A certificate describes the checks on the current evidence. A decision receipt describes an earlier commitment made from evidence. We represent receipt i as $d_i = (\pi_i, x_i, W_i, v_i, a_i, k_i, O_i)$, where π_i is the frozen policy version, x_i is its non-evidence context, W_i is its declared transitive read set, v_i records the versions last verified for that decision, a_i is the historical action, k_i is the preceding checkpoint, and O_i contains registered continuation outputs. The implemented history is linear. Removing its suffix represents control dependence; the implementation is not a general concurrent multi-agent rollback protocol.

The non-evidence context includes the retry counter and retry limit at the original decision. Reconsidering that decision does not spend its retry budget again. A value that may change and influence the decision must instead be in the read set. Freezing an undeclared changing input in x_i would invalidate the consistency premise. A stochastic language-model call is not made deterministic by recording its model name or requesting temperature zero. The guarantees below apply to executable deterministic policies, or to a separately justified deterministic decision adapter.

Write $V_E(W_i)$ for the current input-version vector of receipt i. Its dirty predicate is

$$D_i(E) = [V_E(W_i) \neq v_i]. \tag{1}$$

This comparison is per receipt. A union of all changed paths can safely select more receipts, but may replay a decision whose own current attestation already matches its inputs. That extra work explains the earlier generic-baseline advantage; it is not evidence for a graph advantage.

1.4 Progress-preserving counterfactual branch revalidation

The original store retained refreshed attestations only after an entire replay invocation succeeded. If $m > b$ affected decisions all retain their actions, each invocation with budget b could replay the same prefix and block again. Failing closed prevents an unsupported continuation but does not establish progress. The revision persists each successfully verified unchanged receipt, including a prefix verified before later budget exhaustion. These attestations serve as persistent progress; no separate speculative action is accepted.

In historical order, the algorithm evaluates dirty decisions until it finds the first changed action or spends the invocation budget. For an unchanged action it commits $v_i \leftarrow V_E(W_i)$. For a changed action at j, it persists rollback intent, invalidates receipts $j{:}n$, quarantines the registered $\bigcup_{i \geq j} O_i$, and restores k_j. Missing inputs and unavailable policy versions leave the history blocked. Recording a new decision is disallowed while the store is blocked. Callers must also consult that status before treating existing products as publishable; merely leaving a file on disk does not make it current.

Algorithm 1 Progress-preserving CBR for a persistent linear history

Require: Receipts $d_{1:n}$, current evidence E, invocation budget $b > 0$

Recover a pending rollback; verify policy versions and input versions
$I \leftarrow \{i : D_i(E)\}$ in historical order
for $i \in I$ **do**
  if $b = 0$ or replay is unavailable **then**
    Persist blocked status and previously verified attestations;
    return
  end if
  $a'_i \leftarrow \pi_i(E|_{W_i}, x_i)$; $b \leftarrow b - 1$
  if $a'_i \neq a_i$ **then**
    Persist invalid suffix and rollback intent
    Quarantine registered suffix outputs; restore k_i;
    return
  end if
  Persist $v_i \leftarrow V_E(W_i)$ without changing the original action
end for
Persist unblocked status for the surviving history

A later evidence change automatically makes an earlier attestation dirty again through Equation 1. Consequently, persisting progress does not authorize reuse across an input revision. Our store is a cooperating single-writer, quiescent-input implementation. Concurrent writes during one invocation require a snapshot protocol and are outside the implementation.

1.5 Conditional consistency, progress, and costs

Assumptions. The initial recorded history reproduces its actions. Every action has a complete read set, the policy and frozen context are unchanged and deterministic, and input bytes are stable during an invocation. Registered checkpoints and outputs suffice to restore the declared local continuation. An error returns a blocked state rather than being interpreted as an unchanged action.

Proposition 1: earliest-flip equivalence. On an unblocked completion or rollback, progress-preserving CBR identifies the same earliest changed action as full historical replay on that evidence state. If no action changes, the surviving history is current under the registered contract.

Proof. An unaffected receipt has the same read values, policy and context as its last successful attestation; determinism preserves its action. A dirty receipt can be removed from the work set only after its current action was explicitly reproduced. These statements hold initially and after each committed unchanged receipt. Processing dirty receipts in increasing index therefore skips no possible earlier flip. The first observed flip is also the first flip of full replay. If all dirty receipts pass, every action is either unaffected or explicitly reproduced. A subsequent evidence update invalidates exactly the relevant attestations and re-establishes the same argument for the new state. Rollback removes the control-dependent suffix, so no action in that discarded suffix is asserted to remain valid. □

Proposition 2: bounded-budget progress. Suppose evidence and policy remain fixed between invocations, replay succeeds, and each invocation permits $b > 0$ evaluations. Let r be the number of initially dirty receipts up to and including the first flip, or all m dirty receipts when none flips. Completion or rollback takes at most $\lceil r/b \rceil$ invocations and exactly r replay evaluations, with zero invocations needed when $r = 0$.

Proof. Each unchanged replay permanently removes one member of the dirty set for this fixed evidence state. No such member re-enters. Every nonfinal invocation therefore removes b members, while the final invocation either removes the remainder or observes the first flip. Partitioning the first r evaluations into groups of at most b gives the bound. There is no finite progress bound under endlessly changing evidence or unavailable replay. □

This is an operational specialization of incremental computation and persistent validation, not a new general change-propagation principle [1, 2]. A generic dependency system with the same attestations and rollback interface can realize the same algorithm. The matched generic-progress baseline tests that boundary explicitly.

Let $L = \sum_i |W_i|$, H be the bytes inspected, r the replay count and s the invalid suffix size. The present flat implementation scans $O(L)$ binding entries per invocation, reads $O(H)$ bytes counting repeated reads, invokes r policies, and writes attestations plus up to s invalidations and registered output moves. Its repeated hashing and receipt persistence can dominate cheap policies. Across q bounded invocations the scan cost can be $O(qL)$, even when policy replay remains r. Continuation adds its actual fitting and publication work. No asymptotic index advantage or general wall-time improvement over ordinary selective replay follows from Propositions 1–2.

1.6 Scope of closure and verification errors

For a fixed interpretation and scientific scope of c, let $R^\star(c)$ denote all obligations needed to justify it. Requirement generation may produce an incomplete set, $R(c) \subsetneq R^\star(c)$. Even if the verifier accepts no false represented requirement, a necessary omitted obligation can remain unsatisfied. Excess or malformed requirements create additional utility and specification problems; they are not resolved by the closure rule either.

Let $a_r = 1$ mean that requirement r is CLOSED, $A = 1$ mean that every member of $R(c)$ is CLOSED, and z_r indicate true satisfaction. Define F_{claim} as selecting a claim for which some obligation in $R^\star(c)$ is unsatisfied. Let F_{omit} be the part of this event in which a selected claim has an unsatisfied obligation in $R^\star(c) \setminus R(c)$, and let $F_r = \{a_r = 1, z_r = 0\}$ denote false acceptance of represented requirement r. For a fixed claim and requirement schema, every full-claim failure involves an omission or an accepted false represented requirement. Hence

$$\Pr(F_{\text{claim}}) \le \Pr(F_{\text{omit}}) + \sum_{r \in R(c)} \Pr(F_r). \tag{2}$$

The union bound requires no independence assumption. For varying schemas, it applies conditionally and can then be averaged over them. Component error budgets control represented requirements; the omission term requires a separate assessment of completeness. Conditional selective risk divides the unconditional failure probability by nonzero selection coverage. Our experiments leave omission risk and distribution-free component bounds unestimated, so the certificate remains relative to $R(c)$.

Statistical risk qualification. Risk certification requires bounds valid for the evaluated policy. If $q_{r,T}$ bounds false acceptance of requirement r conditional on the adaptive history at stopping time T, the union bound gives represented-set risk at most ϵ when $\sum_r q_{r,T} \leq \epsilon$. Static calibration does not supply such history-conditional bounds. Alternatively, freezing the entire adaptive episode policy allows calibration of its population selective error on independent episodes, with simultaneous bounds for finite-policy selection [3]. The appendix gives a sufficient construction. Both routes require evidence beyond our data and leave the omission term in Equation 2 uncontrolled.

The prototype checks a historical fixed-policy calibration upper bound against $\epsilon = 0.10$. Calibration fails, so it issues no risk-qualified closure. Candidate closure without the guard is reported separately. The executor reads and extracts existing artifacts; experiment and replication actions remain specifications.

Execution scope. Authenticity and relevance use source hashes, task identifiers, extraction receipts, and literal evidence bindings. A semantic rubric checks the requested metric, dataset and method, row or column, aggregation, context, and unresolved conflicts. Sufficiency and discriminativeness need claim-specific tests for experimental adequacy.

The archived CORE extractor scans admissible files in sorted order for a reference value and keeps its first match. Its comparators evaluate supplied manifest predicates. The acquisition controller instead receives a question and public artifacts, reserving reference values for scoring. We report these protocols separately because the former starts with reference-assisted bindings and the latter must acquire its evidence.

Table 1: Recorded examples from the evaluated studies, selected after inspecting saved outputs. Version and branch changes are controlled interventions; acquisition reads published artifacts. The cases illustrate recovery, maintenance, and a binding failure; they provide no frequency estimate. Source IDs and receipts are in the Technical Appendix's Recorded Case Studies.

Case	Evidence change and TrustScientist response	Paired comparator and boundary
Two-step recovery SVM F1 on Musk1+	Wrong capsule binding and interrupted extraction keep the report OPEN. Rebind, then resume extraction; re-audit closes the unguarded report with the quoted answer 87.5155.	The structured selector also closes correctly after two actions. Risk-guarded ECC remains OPEN because calibration fails.
Shared-source update Three reporting chains	A changed source invalidates all three downstream certificates. Dependency re-audit checks ten affected nodes and removes all stale closures.	The stateful Qwen validator updates one chain, retaining two stale closures. Flat tracking matches the graph; full re-audit checks eleven nodes.
Historical branch flip Two recorded decisions	Replay leaves the earlier REFINE unchanged, but changes the later PROCEED to REFINE. CBR invalidates the later continuation and rolls back before it.	Full replay chooses the same checkpoint. The certificate-only tracker updates predicates but retains the old PROCEED. No replacement experiment is executed.
Wrong-metric binding Random-classifier macro-F1	The file, digest and supplied relation flag pass, although the quoted value belongs to neural-network macro recall. TrustScientist incorrectly closes.	GPT-5.6-Sol keeps the manifest OPEN and identifies the metric and classifier mismatch.

2 Complete Study Results and Development Record

2.1 Evaluation map and recorded examples

Table 1 records selected examples from the original studies. They illustrate mechanisms and failures without estimating natural error prevalence. Detailed original protocols follow later in this appendix.

2.2 Historical branch revalidation

Can replay recover a usable continuation after evidence changes? We execute a scripted analysis agent on Iris, Wine, Breast Cancer and Digits. Three split seeds and three search lengths yield 36 histories with 4, 12 or 24 decisions. Each history fits distinct candidate configurations, checks held-out admissibility and baseline comparison, and produces a table, chart and report. Six paired updates per history give 216 episodes: unchanged content, early and late executed configuration revisions, a training-score substitution, candidate deletion, and a revised baseline. Updates are controlled; the trajectories are executed analyses, not natural autonomous AutoResearchClaw histories.

Full replay, CBR and a generic flat dependency-replay comparator share the policy, verified file versions, receipt checks, output quarantine, checkpoint restoration and continuation executor. The generic planner scans each receipt's read table. Certificate-only refresh updates products without replaying actions. After rollback, the executor refits inadmissible trials, schedules a bounded configuration revision after a failed comparison, and regenerates products. Each replay method executes

Table 2: Executed controlled histories. Exact counts earliest rollback matches on 102 flip episodes; Final counts consistent final actions and products on all 216 episodes. Calls includes replay, receipt verification and continuation policy entries, excluding common initialization.

Policy	Exact /102	Final /216	Calls
No re-audit	0/102	59/216	0
Certificate-only	0/102	114/216	0
Full replay	102/102	216/216	4,334
Generic replay	102/102	216/216	2,709
TrustScientist-CBR	102/102	216/216	2,805

135 continuation fits. A separate scorer recomputes macro-F1 from saved predictions, derives the earliest changed action, and inspects checkpoints, quarantined files and final table/chart/report consistency.

CBR matches full replay at every rollback point, retains all 114 non-flip branches, and leaves no stale final action. Total post-update policy entries fall by 35.3% (95% paired cluster-bootstrap interval $[32.2, 38.6]$); the 12- and 24-decision histories save 36.2% and 37.4%. Two order-balanced timing repetitions show a smaller 10.3% reduction $[9.3, 11.2]$ in post-update wall time, including refitting and file operations. Intervals resample the 12 dataset–split clusters, not individual mutations. The generic comparator matches correctness and uses fewer calls: its per-receipt check skips some current attestations revisited by CBR's union of changed files. These results support selective replay relative to full replay, not superiority over generic dependency systems or ClaimGarden.

The original CORE study remains separate: 405 controlled updates to 45 capsules produce 254 branch flips across 711 historical decisions. CBR matches all full-replay checkpoints with 382 versus 416 replay evaluations; the certificate-only tracker retains 326 changed historical actions. The appendix preserves that protocol, the new executed trace and cost details.

2.3 Certificate revalidation

We compare executable dependency tracking with a stateful, carefully prompted validator on copies of 124 source files underlying 79 archived reporting states from 45 CORE capsules. Of these files, 39 are shared by questions within 21 capsules. Each capsule has a declared dependency schema linking source versions, extraction receipts, bindings, and closure; 55 initial literal contracts pass. Nine controlled events per capsule change or restore a source, receipt, report, or source–receipt pair, or change an unrelated file. The original artifacts remain untouched.

All policies receive the same dependency metadata, changed-file identifiers, checker, and per-event budget, sufficient for full revalidation. Full re-audit skips unregistered-file changes and otherwise checks every node. The stateful Qwen3-8B validator receives its persistent cache, complete event history, and an explicit transitive-invalidation algorithm. The graph policy traverses an index; the flat-table comparator obtains the same affected set by repeated scans. A malformed LLM plan falls back to full re-audit. All 405 formal model requests complete without transport failures or fallbacks.

An independent evaluator reconstructs file bytes after each event and checks the certificates' declared input versions. Each method makes 711 certificate decisions: 332 contracts are valid, 163 initially passing contracts have become stale, and 216 were initially non-passing. Stale acceptance uses 163 as its denominator; valid retention uses 332. Both metrics refer to the version contract.

Dependency tracking matches full revalidation with 47.0% fewer check evaluations (Figure 1).

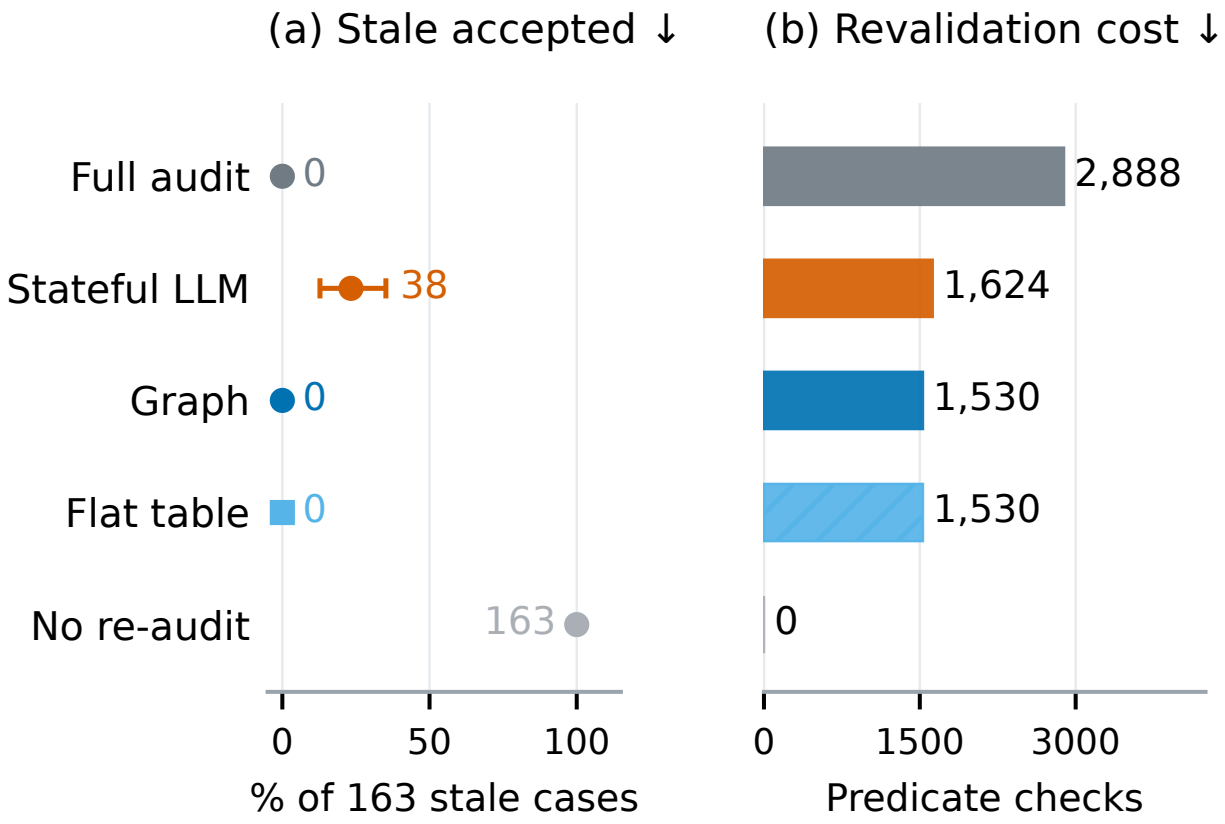


Figure 1: Controlled version updates. Counts beside points are stale certificates accepted out of 163; whiskers show capsule-bootstrap 95% intervals. Graph and flat tracking, like full re-audit, retain all 332 valid certificates; the LLM retains 327. No re-audit retains all 332 valid and all 163 stale certificates. Check counts exclude initialization.

The 95% paired capsule-bootstrap interval is $[42.2, 51.8]$. The LLM retains 38/163 stale certificates (23.3%) and loses five valid closures. Its invalidation set is exact on 227/405 events; 93 events omit required checks. The shared-source example in Table 1 leaves two LLM closures stale.

The flat dependency table matches the graph's decisions and check count exactly. Selective re-audit gains from executing the dependency contract; either data structure can supply the affected set.

2.4 Evidence acquisition

Acquisition can recover unresolved reporting states, but its usefulness depends on both the available actions and the stopping gate. We compare the action selector with a structured LLM using the same tools. The frozen protocol separates natural retrieval difficulties from controlled interacting faults, with reference answers reserved for final evaluation.

Tasks, policies, and measurement. Fifteen capsules, selected by a fixed hash order, are reserved for historical calibration. Evaluation comprises 46 natural questions from 30 capsules and 36 controlled states drawn from 12 nominal-clean requirement bundles in seven capsules. Con-

Table 3: Candidate outcomes before the risk guard, budget 4. Columns show correct and incorrect candidate closures; denominators are all 46 natural or 36 controlled attempts. ECC's final risk-qualified closures are zero. Only w/o risk guard and Structured LLM turn these candidates into final closures. The postcheck-change scenario is included in controlled states.

	Natural ($n = 46$)		Controlled ($n = 36$)	
Policy	Correct	Wrong	Correct	Wrong
ECC	21	9	22	2
w/o graph	21	9	11	1
w/o risk guard	21	9	22	2
w/o budget selection	21	9	22	2
w/o re-audit	18	4	0	12
Structured LLM	21	9	22	2

trolled scenarios pair wrong task binding with an interrupted extraction receipt, pair stale integrity metadata with an interrupted receipt, or change a digest after an initially passing check. The last scenario creates a stale decision deliberately.

All policies share Qwen3-8B, source snapshots, tools, structural checks, and a six-condition semantic rubric. Actions read or search published outputs, refresh a source, repair task binding, or resume extraction. Budgets are 1, 2 and 4 declared units, with at most four actions. Rebinding and resumed extraction cost one unit, search/fetch two, and broad refresh three. Tokens are logged separately from these policy costs. The executor acquires existing artifacts rather than running new science.

The six policies comprise the full evidence-closure controller (ECC), four ablations, and a structured LLM selector. The selector receives the same graph-filtered candidate actions, checks, and prior receipts, and shares the final predicate aggregator without the calibration guard. The graph and obligations are thus supplied to both selectors. Removing the graph replaces it with a flat failure list and fixed scheduling priority. Removing budget-aware selection prioritizes broad refresh and stops when it is unaffordable. Removing re-audit retains cached predicates after evidence changes. The graph ablation also changes ordering.

False termination and successful closure divide invalid and valid final closures, respectively, by all attempts. Selective risk divides errors by closed attempts and is undefined when none close. Recovery success uses initially unresolved predicate states as its denominator. Actions-to-closure and cost-to-closure are conditional means over successful closures; unconditional acquisition cost is reported alongside them. The operational evaluator combines artifact checks, literal binding, and benchmark answer matching. It shares the controller's structural and literal-binding functions. We retain its frozen labels while distinguishing reference agreement from source fidelity: a faithfully quoted output can differ from the reference target.

Calibration and control outcomes. Historical calibration accepts seven reserved capsules, including two automatically scored errors. The one-sided 95% Clopper–Pearson upper bound is 0.659, exceeding the preset 0.10 threshold. The risk guard rejects every candidate. ECC and its guarded ablations therefore produce zero false terminations and zero successful risk-qualified closures, with undefined conditional selective risk. Calibration has failed.

The full factorial contains 1,476 completed policy rollouts, 435 distinct model requests shared through caching, and 807 executed tool actions. Thirteen unusable replies, including three request failures, remain unresolved.

Without the risk guard, budget-4 control produces 21/46 successful natural closures (capsule-bootstrap 95% interval [30.9,59.6]%) and 9/46 false terminations. Controlled outcomes are 22/36 successful closures (interval [52.4,66.7]%) and 2/36 false terminations. The structured selector matches both counts and every paired closure/correctness outcome: 82/82 agree. Natural cases lack source snapshots for restoration, leaving acquisition often limited to repeating a search with the same question at a later offset.

At budget 4, removing the graph reduces correct controlled candidates from 22 to 11. Omitting re-audit leaves 12 stale, invalid candidates and no correct controlled candidates. At budget 2, cost-aware selection produces 11 correct controlled candidates, while broad-refresh priority produces none. The graph contrast includes the changed ordering; the budget contrast includes stopping when the preferred refresh is unaffordable.

Natural recovery reaches 3/24 initially unresolved tasks without the guard. Raising the budget from 2 to 4 adds two false terminations and no correct closures. Saved failures include a processed-data row cited for an analysis-script question and a regression estimate cited as a sample count. Literal quotation misses these context and value-type errors. The appendix distinguishes them from reference-output disagreements and records all budget and cost denominators. These utility outcomes precede the failed risk guard.

Executed trace. The two-step recovery in Table 1 links each executed action to a new evidence digest, receipt, and re-audit. The benchmark-matching candidate closes only without the failed calibration guard; the appendix preserves the complete trace.

Disjoint acquisition pilot. The frozen pilot uses 41 questions from 24 newly acquired public-train capsules, disjoint from earlier cohorts. Both selectors share model-generated hypotheses and probes, accumulated evidence, artifact tools, a four-unit budget, and the final gate. Our heuristic maximizes predicted pairwise disagreement per cost; the validator follows the model's choice. Across 623 requests, each closes 14 correct and 11 incorrect answers. Every outcome matches, and neither repairs any of the 11 initially wrong answers. The heuristic uses 4.4% more tokens.

The validator recommends 59 already executed queries, triggering fallbacks. A post hoc interface repair restricts choices to valid actions. Both policies then close 13 correct answers; the heuristic abstains on one additional wrong answer. The appendix gives the full comparators, costs, and diagnostics.

2.5 Artifact contract enforcement

The archived artifact protocol starts with candidate bindings and applies controlled manifest violations. A restoration test returns each record to its known clean state. A separate fresh-workspace diagnostic reruns public capsules; its outcomes remain separate from the acquisition study.

Requirements and candidate retrieval. We use the 45 capsules in the CORE-Bench test manifest: 18 computer science, 14 social science, and 13 medical science tasks; 22 use Python and 23 use R. Their shipped `results/` directories provide the nominal clean artifacts. Manually authored benchmark questions and reference outputs define the targets. Merging repeated reference executions leaves 79 unique capsule–question requirements. These are benchmark-derived targets, not automatically generated scientific obligations; the evaluator supplies their artifact predicates. The archived human audit forms are unfilled, so neither requirement completeness nor appropriateness has an independent human estimate.

Table 4: Reference-assisted candidate retrieval. Field and language panels describe the same 79 requirements; they are not additive.

Subset	Candidates/total	Coverage (%)
Computer science	29/36	80.6
Social science	21/24	87.5
Medical science	12/19	63.2
Python	34/42	81.0
R	28/37	75.7
Overall	62/79	78.5

Table 5: Matched-manifest compliance on retrieved candidates. Lower corrupted closure is better. Each policy receives the same evidence fields; the LLMs use their saved fixed-prompt configurations.

Policy	Nominal clean closed	Corrupted states closed
DeepSeek-V4-Flash	55/62	149/310
GPT-5.6-Sol	59/62	0/310
TrustScientist	62/62	0/310

The evaluator locates reference values in extracted artifacts and marks the clean records' claim–artifact relations as valid before constructing corrupt controls. Candidate retrieval is therefore reference-assisted. A matching value locates a possible binding, whose semantic accuracy still requires checking.

Candidates are retrieved for 62/79 requirements (78.5%; capsule-bootstrap 95% interval $[66.7, 88.1]$). Table 4 includes all requirements in the denominator, with the other 17 remaining OPEN. Coverage ranges from 12/19 in medicine to 21/24 in social science, reflecting the extractor's coverage of the shipped artifacts.

Matched-manifest decision policies. For each retrieved candidate, five controls change one condition: a missing artifact, an inadmissible type, a path outside the permitted results namespace, a stale digest, or binding to the wrong local question. This yields 62 clean and 310 corrupted states. Only the decision records are mutated; the original bundles are retained.

Both LLM baselines receive the claim, requirement, extracted value, manifest fields, execution status, and semantic evidence supplied to the closure policy. Their inputs include precomputed type, path, digest, and relation checks, but exclude raw files and images, TrustScientist's verdict, and corruption labels. The saved prompt explicitly requires admissible evidence and every mandatory condition to pass, and requests a state, failed condition, confidence, and reason. It makes one joint judgment, without separate AUTH/REL/SUFF/DISC decisions combined by the same external aggregator. Evidence access is matched; the decision procedure is not. This comparison does not isolate typed reasoning from structured prompting and gate enforcement.

DeepSeek-V4-Flash closes 55/62 clean and 149/310 corrupted states (Table 5). GPT-5.6-Sol closes 59/62 clean states and rejects all 310 corruptions. TrustScientist closes all 62 nominal clean states and rejects all corruptions. *Residual corrupted closure* uses every corrupted state in its denominator, independently of whether the policy closed its clean counterpart.

The three GPT nonclosures identify metric, task, or context mismatches. Table 1 shows one: a

value passes the supplied relation flag but belongs to another metric and classifier. TrustScientist accepts that flag and closes incorrectly. Nominal clean closure and correct semantic binding are therefore different outcomes; these GPT abstentions have no established error label.

GPT enforces every tested corrupted-state constraint. A generic validator could also implement the same conjunction. Because the control construction supplies the type labels, their independent inference is outside this protocol.

Restoration and fresh execution. All 310 controlled violations remain unresolved. Restoring each known clean record after naming its first failed gate makes all 310 close on re-audit. This transition exercises restoration under a known intervention.

The fresh-workspace diagnostic covers 20 public-train capsules and 59 task questions. In each of five runs, 19/20 tasks meet their reference targets and 58/59 questions match. Mean summed task wall time per run is 765.9 seconds. Each run starts with a fresh workspace and the same prepared host-specific compatibility repairs. The archived CUDA 10.1 program fails on the Ampere GPU in every run; its CPU fallback also misses the reference target. That task stays in every denominator. The appendix records the per-run outcomes.

2.6 Semantic verification

Paired semantic gates. We hold retrieved context and reports fixed on 79 previously inspected questions while varying the semantic gate, using 237 Qwen3-8B requests. An independent scoring implementation measures benchmark-answer agreement. The literal contract closes 44 correct and 11 wrong answers. A structured rubric closes 35 and four; blind answer agreement closes 34 and six. Adding scope witnesses closes 25 and five. It rejects one error but loses nine correct closures, a success difference of -11.4 percentage points (95% capsule-bootstrap interval $[-19.8, -2.9]$). Additional witnesses reduce useful closure in this paired study. Complete gate definitions and scores are in the appendix.

Fixed-retrieval SciFact. SciFact evaluates relation resolution under fixed retrieval. A pair is CLOSED when its SUPPORTS or REFUTES relation is resolved. All methods receive one sentence selected by a frozen natural language inference (NLI) model, which classifies entailment, contradiction, or a neutral relation. The LLM judge uses `deepseek-chat`, distinct from the manifest baseline. There are 209 pairs with evidence and 130 no-evidence (NEI) controls. TrustScientist correctly resolves 98/209 pairs with evidence and falsely closes 11/130 NEI pairs. Its 118 closures also include nine SUPPORTS/REFUTES reversals.

The no-evidence false-closure rate (NEI-FCR) divides false closures by NEI cases. Coverage divides all closed decisions by all cases, and valid-claim recall divides correctly resolved evidence-bearing pairs by 209. Macro-F1 treats abstention as NEI; selective risk divides all wrong relations by closures. The appendix retains the complete operating-point table and repeated-run comparisons. The matched-coverage curves show the cost in useful relation resolutions that can accompany lower NEI-FCR. The selector hits a gold rationale for 118/209 evidence-bearing pairs (56.5%). On hits, 79 closures contain 78 correct relations and one error; on 91 misses, 28 contain 20 correct relations and eight errors. Eight of nine relation reversals thus occur on rationale misses. This post hoc association is not a causal separation of retrieval and decision errors; the appendix reports the denominators.

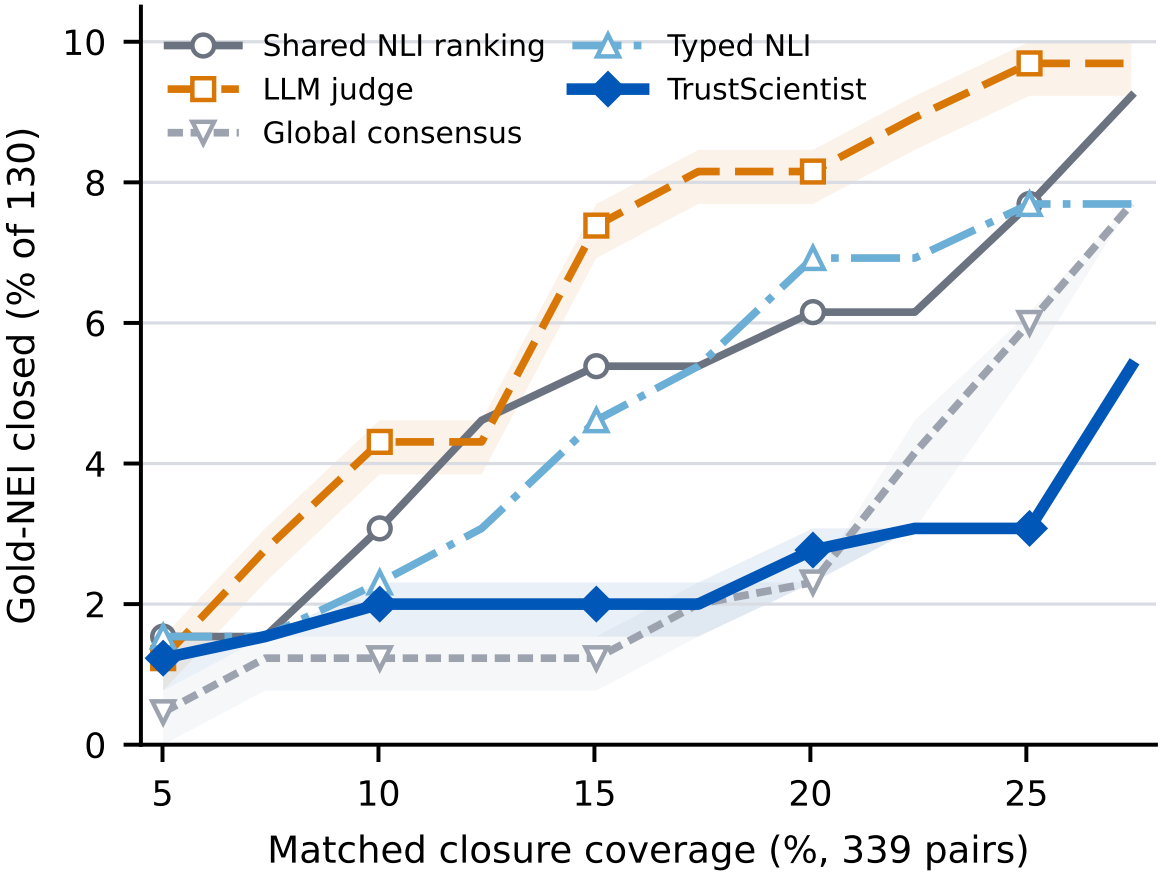


Figure 2: Original fixed-retrieval SciFact matched-coverage curves: five-run means, with shading spanning runs for LLM rankings. The ordinate is closed gold-NEI pairs divided by 130; relation reversals are excluded. Shared NLI denotes direct NLI, rule, and risk-controlled policies. Clipped-score ties retain dataset order, limiting ranking comparisons.

2.7 Consecutive updates and bounded replay

Design and executed work. We freeze the extension protocol before its formal run on 15 October 2026. Iris, Wine, Breast Cancer and Digits are split using three stratified seeds (20261015–20261017). Five or eleven distinct logistic-regression regularization settings yield 12 or 24 decisions per history: a dataset gate, admissibility/comparison decisions for each candidate, and a report gate. The same public dataset families appeared in prior studies; new splits and fits do not create new independent task populations. The classifier fits and saved prediction arrays are real executions in the existing Linux environment (Python 3.11.15, NumPy 2.4.6, scikit-learn 1.9.0; two CPU threads).

Each of 24 persistent histories receives eight consecutive updates: unchanged bytes, late-candidate training-score substitution, restoration, a fitted decision-tree baseline revision, early-candidate substitution, restoration, simultaneous early/late substitutions, and joint restoration. Methods are not reset between events. Interventions are specified by source identity and split metadata rather than selected according to method outcomes. Upon rollback, the common executor really refits inadmissible candidates on the held-out split and regenerates decisions, a CSV

Table 6: New consecutive-update experiment: 192 events per method and arm. Consistent requires completion, current actions and matching products. Calls are all post-update policy entries; all successful replay methods perform 96 actual continuation fits per arm

	Unbounded		Budget two	
Method	Consistent	Calls	Consistent	Calls
Full replay	192	3,304	120	6,336
Generic replay	192	2,532	148	4,832
Original CBR	192	2,532	148	4,832
Progress CBR	192	2,532	192	2,532
Generic + progress	192	2,532	192	2,532
Full + progress	192	3,304	192	3,304
Certificate-only	104	0	104	0

table, an SVG chart and a Markdown report. Unlike the earlier independent-update protocol, this tests newly recorded continuations becoming the history for the next update.

Seven methods share evidence, policies, declared bindings, frozen context, checkpointing, output quarantine and the fitting executor: full replay, generic per-receipt replay, original CBR, progress CBR, generic replay with progress, resumable full replay, and certificate-only refresh. Generic-progress uses a flat per-receipt planner; the progress and rollback interface is intentionally shared. It is a mechanism control, not an independently engineered external system. Resumable full replay retains a cursor valid for a fixed input/policy epoch, removing it when that epoch or the active history changes. In the unbounded arm, all methods have enough replay budget. In the bounded arm, each invocation permits two policy evaluations and each event permits at most 32 invocations. Failed events remain in the denominator and later updates operate on the method's actual surviving store. Histories can diverge after a failure; they are paired by initial history and update schedule rather than by an assumption of identical postfailure state.

Independent scoring. The scorer imports neither the runtime routing function nor its verifier. It recomputes macro-F1 from saved predictions, derives expected actions from metric/split/value fields, determines the first flip against the actual pre-event history, and parses saved per-event tables, chart widths, reports and checkpoints. A completed consistent event requires no stale action and mutually consistent products. An incomplete/blocked event cannot count as successful even if some existing products happen to match. These are contract labels, not expert judgments of scientific merit. All 2,688 records are retained and the independent score recomputation passes.

Results and attribution. With unbounded replay, every replay method completes all 192 events with correct final actions and products; certificate-only refresh does so for 104/192. Each replay method performs 96 actual continuation fits. Progress CBR uses 420 replay evaluations and 2,532 total post-update policy entries, compared with 1,192 and 3,304 for full replay. The total includes receipt verification and continuation. The paired 12-cluster bootstrap saving is 23.4% (95% interval [21.1,25.7]). The other selective methods tie these work counts in this schedule, including original CBR. Its union-of-paths overhead in the earlier experiment is not universal.

Under budget two, progress CBR, generic-progress and resumable full replay complete all 192 events. Original CBR and restarting generic replay complete 148; restarting full replay completes 120. Failed methods remain blocked at the 32-invocation cap, rather than being called correct. Successful prefix attestations eliminate repeated replay work in the fixed-evidence intervals. The generic-progress tie demonstrates that the benefit belongs to preserving validation progress, not

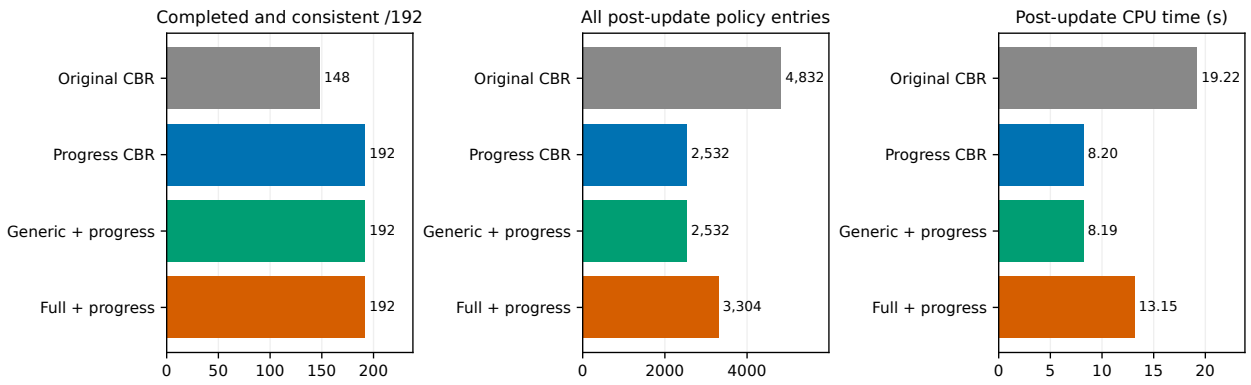


Figure 3: Consecutive-update results with two replay evaluations per invocation. All methods have the same 192-event denominator. Generic-progress ties progress CBR in completion and deterministic work; measured CPU time is descriptive. Total policy entries include receipt verification and continuation

a unique evidence-graph representation. The matched resumable full baseline also succeeds, with higher replay work. This is a bounded-budget implementation result, not a claim that full or generic replay cannot be made resumable.

Cost boundaries. The unbounded total elapsed times are 7.31 s for full replay, 6.95 s for progress CBR and 7.00 s for generic-progress. In the bounded arm they are 13.15 s for resumable full replay, 8.20 s for progress CBR and 8.19 s for generic-progress. Method order rotates across histories; these are descriptive single-host CPU measurements without repeated timing trials. Cluster intervals for workload variation do not estimate machine-to-machine or repeat timing uncertainty, so wall-time generalization is not claimed. The bounded selective method still makes 16,712 file-hash calls over 85.0 MB of counted read bytes and 3,028 JSON writes, versus 30,256 calls, 157.5 MB and 3,918 writes for resumable full replay. Common history initialization costs about 0.81 s and 864 policy entries per method/arm and is reported separately. Preparation fits, snapshots, intervention writes and independent scoring are excluded from post-update times. There are no new foundation-model calls, API charges or token savings.

Ablations and direct functional checks. The original/restarting arms ablate partial-progress preservation while retaining the same tools and policies. Certificate-only refresh ablates action replay and retains 216 stale action instances in each arm. Direct executable tests separately demonstrate progress after a process restart; invalidation of a previously attested earlier receipt when evidence changes while blocked; suffix-only output quarantine; blocking on missing evidence and policy-version mismatch; and a spurious flip when live context replaces the frozen original context. These are mechanism tests, not additional scientific experiments or population-level rates. We do not reuse the coupled acquisition graph and ordering ablations as clean evidence for a graph-specific contribution.

2.8 Generated-analysis pilot on eight existing research projects

Scope and frozen inputs. We additionally ran eight scoped reanalysis tasks using raw data from eight existing public CORE-Bench research capsules: cardiac structure, mobile visual exploration, memory responses, registration discrepancies, breast-cancer omics, CRP meta-analysis, metabolic

classification and news-label prediction. This convenience sample was selected for data/runtime availability before agent outcomes. These are not official full-capsule reproduction scores or eight new end-to-end scientific studies. Source DOIs, raw-input hashes and public/private splits accompany the executed evidence.

Qwen3-8B in BF16 on an existing RTX A6000 generated and executed actual Python analyses for three agent seeds (20261015–20261017). The controller uses three fixed checkpoints (method preparation, execution/interpretation, release), with real code-error repair and update-triggered continuation. Fixed checkpoint records, including initial REFINE actions, are not unrestricted natural research trajectories. Generated code runs without network or private labels in isolated read-only public inputs and a writable output directory. Each task–seed has one measured common prefix, cloned for no replay, resumable full replay, generic selective replay with progress, and CBR with progress. All share the model, tools, cache capability, dependencies, continuation interface and budget: two replay evaluations per invocation, at most eight invocations, three code attempts per continuation and 120 seconds per tool attempt.

Sequential interventions and integrity repairs. The three controlled updates add an unrelated note, expand actual data from approximately 70% of groups/training records to all released records (two to four studies for the small CRP task), and require uncertainty reporting. Validation/test partitions remain fixed. A first frozen run produced 302 of 312 planned snapshots; one task–seed aborted and the ten missing cells remain failures in the planned denominator. Audit found a nonfinite-JSON transport failure, a literal category error in the separately implemented count oracle, leaked cost wrappers after a worker failure, and code regeneration that could stale a retained prefix. A corrected scorer is separately versioned; the original evaluator and all first-run evidence remain unchanged.

The first run also showed that an LLM can disregard explicit stale-execution hashes: after data expansion, each replay method achieved current execution and numerical agreement in only 2/24 planned histories. A second, separately frozen run repeats every task, seed and method. Besides transport/isolation repairs, every replay method uses the same mandatory mechanical artifact gate: failed execution, mismatched input/code/requirement hashes or a missing required uncertainty object yields REFINE. The original model judgment is retained. Continuation executes the dirty analysis producer and actually revalidates any retained prefix whose inputs changed. These are explicit system-interface repairs, not an exclusive CBR innovation or a prospectively planned ablation. Between-round differences cannot identify an algorithm effect.

Independent numeric checks and limitations. Six tasks use input-derived mathematical contracts; two additionally score saved predictions against private dataset test labels that were never exposed to the agent. We separately check whether self-reported validation scores equal those computed from saved predictions. These declared-contract checks are not expert scientific ground truth. Table 7 keeps the full planned 24-history denominator at each substantive update; human scientific validity remains unlabeled. An object called uncertainty does not validate a bootstrap method. Thirteen news/seed-20261015 snapshots lack scorable predictions: their 26 validation/test metric errors remain explicit failures in the denominator. Code inspection confirmed defects in at least 12 of the 72 final requested-uncertainty snapshots across the three replay methods; nine nevertheless pass the numerical-plus-current endpoint and all twelve release PROCEED. Defects include validation-label refitting, reported/saved metric discrepancies, repeated identical resamples, label-as-index errors, incorrect F1 averaging and bootstrapping a different statistic. These retained defects are not independent human labels, and the other sixty cases are not thereby validated. No

Table 7: Second-run pilot: independent numerical agreement with current execution; the final column checks active receipt bindings only. Human validity is pending.

Method	Data update	Requirement update	Current receipts
No replay	0/24	0/24	0/24
Full + progress	16/24	13/24	24/24
Generic + progress	16/24	12/24	24/24
CBR + progress	16/24	14/24	24/24

Table 8: Actual incremental pilot cost. Tokens are input/output; replays include continuation-prefix verification.

Method	New calls	Tokens	Tools	Replays
No replay	0	0/0	0	0
Full + progress	212	275,976/38,404	57	109
Generic + progress	212	279,612/41,278	57	85
CBR + progress	211	274,055/38,076	56	85

selected outcome was replaced by a repaired rerun.

The generic and CBR selectors implement equivalent affected-receipt selection. Within-workspace exact-request caching makes a saved response replayable; new concurrent requests can differ even at equal seed and temperature zero. Cross-method outcome variation therefore does not establish an algorithmic advantage. The independent units are eight task clusters, not hundreds of events. Descriptive task-cluster bootstrap intervals and all per-task outcomes are distributed with the evidence. The news split has four exact train–test statement overlaps; frozen scores and a secondary overlap-excluded sensitivity analysis are both retained. Near duplicates and pretraining contamination are not ruled out. The CRP analysis has only four studies.

Costs and human assessment. Table 8 reports real new model calls and tool executions after the shared prefix; cache hits are not new inference. Additional prefix revalidation is included in replay counts. The separate common prefix costs 104 model requests, 109,256/19,002 input/output tokens and 32 tool executions. Inclusive tables allocate that same measured prefix to each method, without claiming it was physically executed four times. Measured event wall times, request latency, syscall-return bytes, receipt hashing/writes, clones and final persistent storage are archived. Latency sums under concurrency are not exclusive GPU time; traced reads/writes are not physical disk traffic. Controller I/O counters exclude uninstrumented fingerprint/receipt reads; event wall time includes their work. Snapshot copy is measurement overhead outside event timing.

GPU0 power integration over the two formal windows is 0.071834 and 0.090818 kWh, respectively. This includes idle board power and mixed methods, excludes whole-host and unmeasured setup/startup energy, and is not allocated exactly per method. GPU pricing, depreciation and human labor rates are unavailable; no dollar total is fabricated. The original outer-process CPU/RSS counter measures the tracing launcher, not the analysis process; its RSS is not an upper bound. Original analysis CPU/RSS cannot be reconstructed from those counters. Separately labeled saved-code reruns measure direct Python resources; they do not replace original timing or outcomes. The cost coverage report records these limits and the additional measurements. All first-round failures and costs remain separate from the repaired comparison.

The blind packet contains 72 shuffled cases: eight initial cases and four methods at two substan-

tive updates for the first agent seed. Two independent forms request 144 case judgments on binding, method, claim support, uncertainty and release action. They contain no method key, automatic correctness label or private test labels. All human judgments are pending; AI integrity review is same-family and provisional. The pilot does not certify scientific truth, requirement completeness or natural-history utility.

2.9 Verified incremental redecision with a matched fine-grained comparator

Mechanism and explicit reuse scope. We implement an additive decision-context layer without replacing the original CBR interface or figures. A receipt binds the task, question, frozen policy, analysis-implementation AST, requirements and declared dependency facts. For each method, interpretation and release obligation, the shared guard permits reuse only if the bound dependency projection is unchanged and the previous structured conclusion satisfies the finite public contract. Otherwise the delta compiler transmits current numerical operands, uncertainty and requirements, and may replace other unchanged support with an equality witness. The full support remains retrievable; all comparators can request it under the same three-call limit. A byte-length heuristic falls back to the dependency slice when witness bookkeeping would be larger. This is not a guarantee of a minimal token representation. Numerical reports are rendered from current executed artifacts, never an old natural-language summary.

Stability alone cannot certify scientific truth. The guard covers the explicitly implemented finite interpretation fields; complete requirements and arbitrary natural-language conclusions are not proven. This specializes established dependency-based reuse, and a generic system using the same context compiler can implement the same behavior.

Shared statistical repairs. In a separate, versioned regression corpus, all twelve previously confirmed defective snapshots receive a shared audited analysis implementation; originals remain unchanged. Repairs keep fitting on training data, preserve point estimates, use the requested statistic and sampling unit, initialize the RNG once, and bootstrap fixed validation predictions rather than refitting on validation labels. All twelve repair copies pass independently implemented raw-input point/prediction checks and bootstrap recomputation. Fifty-eight controlled fault mutations are detected. Three additional self-consistent but wrong-width bootstrap counterexamples are rejected using the raw-input sample size and RNG schedule. These are engineering regression tests, not evidence that CBR autonomously discovered the repairs.

Prospective comparison after preflight. We retain the same eight public-data reanalysis tasks and use three agent seeds (20261018–20261020), four methods and seven controlled changes: an unrelated note, data expansion, an uncertainty requirement, a cosmetic source-context comment, an explicit request to assess population/causal interpretation, a deliberately corrupted statistical artifact, and actual repaired reexecution. The formal plan has 672 method–event cells and 24 common initial cells. Every method uses the same manually implemented statistical recipes, independent public verifier, Qwen3-8B BF16 service, obligation granularity, cache capability and context-expansion interface. The three fine-grained methods also share reuse predicates; complete review bypasses this guard. Thus this study measures decision maintenance after actual data computation, rather than autonomous generation of a new scientific analysis. The strong comparator uses the same guard and dependency-closed context slice. Complete-context review is an additional comparator; exact-request caching remains enabled for all methods. Controlled corruption reverses interval endpoints (or adds an unsupported population interval to descriptive counts), and is labeled separately from naturally occurring analysis failures.

Table 9: Frozen incremental-decision comparison. Correctness is a finite numerical/uncertainty and structured-interpretation contract plus a narrow interval-percentage text screen, not human scientific validation. Tokens/calls include an equal allocation of the same measured common prefix.

Method	Contract	Fault rejection	Total tokens	Calls
Full review	124/168	24/24	1,265,513	360
Fine + full context	125/168	24/24	1,039,755	297
Fine + slice	159/168	24/24	451,303	328
Verified delta	162/168	24/24	433,995	328

Relative to fine + full context, the observed total-token reduction is 58.26%, with a descriptive eight-task cluster interval for the saving fraction of [55.57%, 61.38%]. Relative to fine + slice, the observed total-token reduction is 3.84%, with a descriptive eight-task cluster interval for the saving fraction of [3.51%, 4.18%]. The resampling units are eight developmental task clusters, not seeds or events. Both input and output token usage, failed requests, cache hits and context expansions are retained; common work is counted once in the actual physical-run total. The comparison does not identify a CBR-exclusive capability, predictive-model improvement, unrestricted scientific-agent success or SOTA. The common initial stage satisfies the finite contract in only 10/24 cases; its fourteen failures are preserved and cloned identically. Verified delta still has six mismatches released as PROCEED: three uncertainty-status errors and three negative point directions labeled zero. These remain failures. The status convention is not itself a scientific validity label: for CRP, analytic confidence bounds are already requested in the initial point schema, so some available/not-requested disagreements in other methods are ambiguous. When the uncertainty-status field is ignored only for events that did not request uncertainty, both verified delta and fine + slice match 165/168 cases. This explicitly secondary sensitivity does not rewrite the primary scores or validate scientific text; the three-record primary difference is not evidence of improved scientific reasoning.

A separate post-hoc endpoint-sufficiency diagnostic constructs the finite structured judgments directly from public statistical artifacts and verifier status, without new policy-model calls. It reuses already measured analysis and verification, whose costs are not zero; it is not a fifth prospectively frozen primary method. This demonstrates that the finite score itself does not require an LLM. It cannot establish scientific reasoning or the necessity of model review, and free-text scientific assessment remains separate.

Auditing and remaining evaluation. Direct analysis and verifier CPU/RSS are measured by GNU time inside their sandboxes; API usage, tool wall time, storage and GPU-board power are archived. The logical-byte counter is partial and excludes prediction CSV reads in the verifier, controller reads and physical disk traffic. Backend cached-token counts, controller-wide resources, whole-host/setup energy, prices and human labor remain unknown. A numerical-tolerance boundary can make a near-zero cached sign unsafe in the frozen implementation; a separate hardened guard abstains from that reuse. Exhaustive offline replay checks the patched guard against the recorded frames without relabeling it as new inference. An additional shared release-layer patch blocks unsupported typed conclusions and retains the raw model judgment. Paired post-hoc replay blocks the six delta mismatches without rewriting the primary scores or counting abstention as successful completion. This is not a new inference cohort or holistic scientific verification.

An additional blind packet contains 104 shuffled first-seed cases: eight common initial cases and every method at data, uncertainty and scope changes. Two independent reviewers are asked for 208 judgments. No method key, automatic correctness label or private test label is included; every

Table 10: V3 failed batching optimization. Release and joint (all three obligations) are public finite-contract conformance with a narrow text screen, not scientific accuracy. Each arm pays its own actual initialization.

Method	Release	Joint	Total tokens
fine guard slice	164/192	164/192	516,604
fused slice	124/192	101/192	224,738
generic region	131/192	107/192	205,652
CBR compiled	145/192	124/192	183,602
symbolic contract	192/192	192/192	0

human judgment is pending. A synthetic preflight model response mislabeled a 95% interval as 90%; that record is preserved and motivates the separate text screen. Passing that narrow screen does not validate all numerical statements or scientific interpretations. The existing generated-code pilot and its failures remain part of the paper.

2.10 Token optimization: failed batching and certified finite dispatch

We preserve a failed optimization rather than treating its lower token count as an improvement. V3 independently executed each method's initialization, eight tasks, three new agent seeds (20261021–20261023), five methods and eight states (960 records). All model arms shared Qwen3-8B BF16, temperature zero, JSON-schema output constraints, evidence retrieval, numerical verification and the release gate. The principal generic region comparator shared the candidate's fine-grained obligations, finite-region proof reuse and joint calls. Compiled delta additionally shortened input aliases and replaced unchanged support with verified witnesses; the compiler is applicable to generic systems.

Compiled delta uses 183,602 tokens versus 205,652 for generic region (10.72% observed reduction; descriptive eight-task cluster interval [-58.40%, 38.91%]). There are 17 paired regression endpoints: 12 release and five joint, over 12 unique task/seed/event states. Its release result is also lower than the separate-obligation strong comparator (145/192 versus 164/192). Thus V3 fails the predeclared no-paired-loss condition and is not an accepted optimization. The symbolic contract control needs no policy-model tokens; it exposes the finite endpoint's lack of evidence for model necessity. All failed setup attempts and raw responses remain archived. V3 consumes 1,130,596 formal and 42,162 setup model tokens across its arms.

Certified dispatch and fallback. V4 therefore restricts its fast path to an explicitly registered finite contract. The registry binds the exact question, analysis recipe and semantic implementation hash, requirement schema, allowed scopes, rule source and verifier source. Independent numerical checks must pass, and near-zero predicate boundaries are excluded. An unchanged binding and decision digest permit exact receipt reuse; otherwise the finite rule is recomputed. Reports always render current verified values and newly bounded explanatory text. No certificate is asserted for arbitrary scientific prose.

Unknown questions, requirements, scopes, recipes, changed implementations or numerical boundaries invoke the separate-obligation model reviewer, without reusing an unregistered prior decision. An actual synthetic unknown-question API witness verifies this route and its accounting, not scientific adequacy. Mechanical verifier failures remain rejected without a policy-model call. The matched generic certified baseline receives exactly the same registry, rules, release gate and

Table 11: V4 registered finite dispatch: eight tasks, seeds 20261024–20261026, two methods and eight states (384 records). The rule defining these public finite endpoints is also available to both methods; this is conformance, not independent scientific ground truth.

Method	Release	Joint	Policy tokens
generic certified	192/192	192/192	0
CBR certified	192/192	192/192	0

fallback; it recomputes the finite rule each time. Any system with this capability can obtain the same bounded behavior.

With a sound verifier, faithfully registered rules, trusted controller-owned receipts and immutable checked artifacts, the fast path returns the registered finite decision. Exact reuse additionally requires equality of all bound fields, the registry and the saved decision digest. This digest is a corruption check, not authentication against an adversary altering both receipt and digest. These are conditional conformance properties, not new proofs of scientific truth, policy completeness or general explanation equivalence.

Both methods pass 168/168 valid states and reject all 24 injected faults per arm. Each performs 96 real analysis executions and 192 independent verifier executions. The formal policy-token total is zero for both arms: this is a tie, with zero absolute token difference and an undefined percentage-saving denominator. Setup and the real fallback witness consume another 2,498 model tokens. Zero policy tokens do not mean zero computational or scientific review cost. Direct tool CPU/RSS, dispatch CPU subsets, logical bytes, storage, failures and all observed usage accompany the results. Physical I/O, backend cached tokens, whole-controller CPU/RSS, whole-host/startup/development energy, hardware price, human costs and development-assistant tokens remain unknown.

The bounded finite acceptance condition passes, but strict token advantage over the equally capable generic baseline is absent. V4 reuses the same developmental cohort after observing V3; new agent seeds do not supply new independent studies. A fixed 64-case packet covers both V4 arms, all eight tasks, the first seed and four substantive states; its 128 human judgments are blank. It can assess the appropriateness of these reports but cannot establish noninferiority to free-text model science. Scientific noninferiority, CBR-exclusive advantage and SOTA remain unestablished. V2's earlier results, the original narrative and all ten original figures are preserved.

2.11 Repairing an omitted requirement and rerunning matched controls

The finite endpoint in Section 2.10 did not establish that every requested scientific quantity was present. A subsequent model review exposed a concrete shared omission: the two classification tasks requested accuracy and macro-F1 against a majority-class dummy, but both implementations emitted only the dummy's macro-F1. The registered contract also omitted dummy accuracy. Thus a perfectly reproduced decision could still accept an incomplete analysis. We retain the earlier results with their original scope rather than retroactively treating them as complete.

Shared repair. The analysis tool now emits validation accuracy, validation macro-F1, dummy accuracy and dummy macro-F1. The dummy class is selected exclusively from training labels and evaluated on the same validation rows as the trained classifier. A separately implemented verifier recomputes accuracy from saved row-aligned predictions, and checks the train-only majority even when model refitting is disabled. The common artifact contract and release gate reject missing, non-finite or Boolean metric values; all three arms receive these checks and identical metric-specific

Table 12: The repaired classification outputs (controller seed 20261027, CBR arm; the principal paired analyses are identical). Accuracy is measured against dataset labels on the fixed validation split. These classifier metrics are distinct from decision-contract conformance.

Task	Data state	Accuracy	Dummy accuracy	Macro-F1	Dummy macro-F1
Metabolic	Initial	0.94293	0.58313	0.85138	0.08185
Metabolic	Expanded	0.94541	0.58313	0.85224	0.08185
News	Initial	0.60352	0.56152	0.53544	0.35960
News	Expanded	0.60840	0.56152	0.55387	0.35960

Table 13: V5 after the shared dummy-accuracy repair. Release and joint conformance coincide. The endpoint is a public finite contract, not scientific ground truth. All 192 planned records per arm are present.

Method	Contract correct	Injected faults rejected	Policy tokens
Generic finite dispatch	192/192	24/24	0
CBR certified	192/192	24/24	0
Fine guard slice (model)	153/192	24/24	505,444

instructions. An unknown registered contract retains actual model advice for inspection but returns REFINE, never an unregistered certificate. Ninety-eight development and mutation checks pass after a preflight repair: an initial non-finite-value test failed because a receipt serializer rejected NaN. The repaired adapter records the invalid value separately and saves a strict-JSON rejection. The failed attempt and its cost remain archived. These are development tests, not additional benchmark observations. The archived public task descriptions still contain two legacy `estimate_keys` lists omitting dummy accuracy. The frozen V5 runner metadata, common required-metric contract and executed outputs contain all four metrics. This documentation inconsistency is disclosed in an erratum; the archived inputs are preserved rather than silently rewritten.

Frozen comparison and cost boundary. V5 freezes source, inputs and the review protocol before execution. It uses all eight developmental tasks, three new controller seeds (20261027–20261029), eight states and three arms, yielding 576 records. The states are initialization, an unrelated note, expanded data, an uncertainty requirement, a cosmetic code change, a scope request, controlled numerical corruption and repaired execution. Each arm pays for its own initialization, four real analyses and eight real verifier calls per task/seed. Arm order rotates across jobs. All numerical evidence, requirements, release checks and decision granularity are shared. The principal comparator is generic finite-rule recomputation with the same registered rules and conservative fallback as CBR; a secondary fine-guard-slice comparator calls Qwen3-8B separately for changed obligations. The latter has the same evidence and safeguards but a different decision implementation. Its contrast measures finite dispatch versus model deliberation, and cannot isolate a CBR-specific contribution.

The principal pair agrees on all 192 analyses, complete decision triples and rendered reports. Both pass all 168 valid states and reject all 24 injected faults; neither makes an unsupported finite release. The secondary comparator is correct on 129/168 valid states and all injected faults, for 153/192 (79.69%) overall. Its errors are conservative failures to match the represented valid-state contract, not observed unsafe finite releases. These frequencies describe controlled, dependent developmental states; they are not estimates of general scientific accuracy. Reusing task families

Table 14: Measured V5 costs, including each arm's initialization. Wall seconds are sums of direct tool executions and can overlap across jobs. CPU and RSS cover direct tools, not the entire controller or model service.

Measure	Generic certified	CBR certified	Fine guard slice
Actual analysis/verifier calls	96/192	96/192	96/192
Analysis wall seconds	105.08	105.46	106.65
Verifier wall seconds	196.59	195.35	196.29
Direct user+system CPU seconds	338.76	337.30	339.58
Peak direct RSS (KiB)	330,624	331,396	330,612
Actual policy API requests	0	0	345
Prompt/completion tokens	0/0	0/0	465,293/40,151
API latency sum (s)	0	0	1,029.82
Persistent workspace (MB, decimal)	215.67	215.67	219.35

after earlier diagnostic results precludes a held-out generalization claim, despite the new controller seeds.

CBR reuses 48 exact decision proofs and computes 120; the generic control computes 168 and does not enable certificate memoization. This control isolates finite decision capability, not the strongest possible selective replay implementation. Ordinary memoization can reproduce this reuse, so it is not an exclusive algorithmic advantage. Both still perform every numerical verifier call. Their absolute token difference is zero and the percentage reduction is undefined because the comparator's denominator is zero. The secondary model arm records 138 application cache hits, no API errors, no parse errors and no missing usage. Its 505,444 formal tokens plus 2,597 shared setup tokens give 508,041 measured local-model tokens for V5. Setup includes the real unknown-contract fallback witness and the failed preflight attempt. No unexecuted model calls are charged as actual usage.

Sampled GPU-board energy during the formal window is 0.04933 kWh across 324 readings. It excludes service startup, the overnight pre-run idle period, development and whole-host energy, and is not attributable to individual arms. Physical disk I/O, backend cached tokens, complete controller CPU/RSS, hardware pricing and human time remain unavailable. The separate GPT-6 review and development assistant do not expose token or charge telemetry here; their cost is unavailable, not zero. Thus 508,041 is the measured local-model total, not the total cost of the research project. Raw requests, responses, failures, timestamps, CPU/RSS, logical byte counts, saved predictions and frozen hashes accompany V5. Logical byte counters omit some verifier prediction-file reads and are therefore incomplete logical I/O measures. The recorded dispatch CPU subset is 0.19034 seconds for generic recomputation and 0.19088 seconds for CBR, providing no measured CPU advantage. The original analysis environment is Python 3.11.15, NumPy 2.4.6, SciPy 1.17.1 and scikit-learn 1.9.0. Registry semantic hashes depend on Python's AST serialization; upgrading Python can change the hash of unchanged source. Reproduction must preserve that environment or explicitly version the serialization. A local Python 3.12 check differed because of newly emitted empty AST fields, not because the archived source bytes changed.

Prespecified GPT-6 scientific assessment. Before formal execution we selected all eight tasks, the first controller seed and four substantive states (initial, expanded data, uncertainty and scope) for both CBR and the secondary model arm: 64 reports. Two fresh GPT-6-Astra contexts at ultra reasoning effort each assessed every report on numerical interpretation, statistical method, uncertainty, evidential scope, release action and explanation. Any invalid dimension makes the

Table 15: Actual GPT-6 model judgments of 64 preselected V5 reports. V/I/U means valid/invalid/uncertain. These are same-family model-assessment proxies, not scientific ground-truth accuracy or human evaluation.

Report source	Reviewer 1 V/I/U	Reviewer 2 V/I/U	Both valid
CBR certified	30/0/2	30/0/2	30/32 (93.75%)
Fine guard slice	24/6/2	22/6/4	22/32 (68.75%)

overall rating invalid; otherwise any uncertain dimension makes it uncertain. All six must be valid for an overall valid judgment. The primary descriptive endpoint is agreement of both reviewers on valid. Reviewers had raw public data, saved predictions, bootstrap indices and replicates, original worker versions and provenance. Method names and automatic scores were withheld and review order was independently shuffled. Output style can still reveal the implementation; access separation was instructional, not enforced by an independent security boundary. Both reviewers' filename inventories exposed optional human-template filenames, but neither read their contents. No human labels were created or inferred.

Both reviewers independently reproduced all 64 estimate sets, 24 full bootstrap traces and 16 saved-prediction cases. They agreed on 62/64 overall labels (96.875%; Cohen's $\kappa = 0.896$); agreement does not establish correctness. Of 32 paired states, both implementations receive two valid judgments in 22, only CBR does in eight, and neither does in two. No pair favors only the model arm under this descriptive endpoint. The generic finite-dispatch arm has identical rendered report content to CBR in all 192 principal pairs; it was not independently rated a third time. The review therefore supplies no CBR-specific quality advantage over that control and no scientific noninferiority guarantee on unseen tasks.

The two unresolved CBR cases are the omics uncertainty and scope states. Their gene-pair Spearman estimates and resampling arithmetic reproduce, but the data do not establish gene-row exchangeability or dependence. The percentile interval is therefore not independently justified as calibrated scientific uncertainty. Descriptive association remains available, while the stronger inferential interpretation remains unresolved. The shared finite contract did not detect this scientific adequacy gap.

The reviewers agree on six invalid secondary reports: a null-crossing odds-ratio interval is described as excluding one; a macro-F1 interval relative to zero is used to justify comparison against the dummy; three descriptive count reports are blocked without a scientific reason; and a walking report imports an unrelated preregistration rationale. Two further secondary reports contain ambiguous "significantly higher" wording, which one reviewer accepts descriptively and the other marks uncertain. All original reports, judgments and disagreements are retained; no outcome-based rerating or adjudication was performed. Reviewer-only calculations do not replace an incorrect argument in the original report. The same-family assessment is provisional, with no human validation and no claim that 93.75% estimates true scientific accuracy. This finite, reused developmental sample supports diagnostic observations, not SOTA.

3 Generated Agent workflows: protocol, failures and costs

Archive naming. WF1, WF2 and WF3 denote the generated-workflow rounds of 19 October. Their frozen archive names retain R1, V2 (also called R2 in logs) and V3, respectively. The earlier optimization V3 is a separate study. Supporting package S3 contains the workflow rounds; S2f contains the earlier V3. The Submission Index provides the complete filename and checksum

mapping.

Study boundary. The 2026-10-19 study uses six previously prepared public research projects, three seeds (20261019–20261021) and a local Qwen3-8B actor. It is a development convenience sample, not a held-out benchmark or an official full CORE-Bench score. The actor writes actual Python, executes it in a restricted workspace, receives actual errors and has up to three code attempts. It also generates the narrative report. Orchestration, update schedules, the release rule and the downstream worker are specified by the experiment. This is a bounded research workflow, not an unconstrained autonomous scientist.

Inputs and numerical checks. The actor sees public CSVs, task instructions, schema and current metric requirements. WF3 additionally gives observed row/column counts, missingness, sample rows and low-cardinality category counts, plus its own stdout and submitted analysis in repair feedback. These are actual observations, not reference answers. Each execution mounts a fresh read-only input copy; verifier targets, verifier outputs, old analysis and host files are excluded. An independently written calculation recomputes the requested estimates. The metabolic task also checks validation and test-feature predictions from the specified train-only model, including dummy predictions and both dummy metrics. This verifies specified computations, not the suitability of every scientific assumption. Frozen hashes bind source and prepared inputs; per-attempt manifests record copied input hashes and were audited against them. Preparation scripts and original provenance remain in the earlier pilot archive. Some prepared data do not distinguish alternative computations that coincide numerically, such as pooled and participant-weighted response rates when group sizes agree. The checks do not validate those alternatives on distinguishing inputs.

Actual continuation and update sequence. The Agent produces a Markdown narrative, a JSON report and an exported JSON result. A SQLite queue holds jobs referring to the report digest and estimates. A real local worker consumes the jobs and writes a downstream artifact. For metabolic classification it computes accuracy and macro-F1 gains over dummy; for other tasks it orders estimates and counts reported metrics. Before each change a new job using the old report is queued. The consumer intentionally has no second freshness gate, making failure to withdraw that job observable. Effects are local and reversible. We do not roll back sent messages, external service jobs or physical operations.

Each history first replaces initial data with expanded public data, then adds the task-specific metric, then executes a defect adding 0.25 to its first required estimate. The added metrics are mean difference (cardiac), median paired difference (walking), pooled response rate (memory), coding-row count (preregistration), Pearson correlation (omics), and balanced accuracy with dummy balanced accuracy (metabolic). The changes are explicitly controlled.

Fair engine and interface controls. TrustScientist's existing persistent store is compared with a separately written generic engine that does not import it. Both use identical release checks, dependencies, output resources, checkpoint and continuation code. Each is crossed with explicit per-path registration and the same shared directory-ownership adapter. The primary engine comparison gives both the managed adapter. A no-recovery diagnostic executes the stale continuation. Exact request-payload caching gives paired arms the same Agent responses; analysis tools still execute. Order rotates by task and seed. Caches do not provide an equal-cache-warmth wall-time comparison.

Table 16: Controlled omissions in WF3, per engine. TrustScientist and the independent generic engine have identical outcomes. Entries count safe withdrawals; old jobs executed are shown in parentheses.

Probe	Explicit paths	Owned directory
Omit report	0/18 (0)	18/18 (0)
Omit export	0/18 (0)	18/18 (0)
Omit queue	0/18 (18)	18/18 (0)
Omit follow-up	0/18 (0)	18/18 (0)
Omit data dependency	0/18 (18)	18/18 (0)
Queue outside owned root	0/18 (18)	0/18 (18)

The planned WF3 denominators are 18 initial histories, 270 normal method–event records ($18 \times 3 \times 5$), and 432 omission probes ($18 \times 6 \times 4$). All are observed. Four recovery methods each pass 54/54 updates and run zero stale jobs; no recovery passes 0/54 and runs 54 stale jobs. Success requires withdrawal before consumption, the analysis checkpoint, current output bindings, required numerical checks, and independent downstream arithmetic. It does not judge the report's complete natural-language argument.

Per-output omissions delete manual registrations; the managed arm has no such statements and tests containment of those resources instead. A new unregistered input filename is not tested. Thus these probes demonstrate specific integration hazards and a directory boundary, not natural error rates, identical code mutations, or automatic discovery of dependencies. There are six task families, not hundreds of independent scientific tasks.

Concrete trace. For metabolic seed 20261019, validation accuracy changes from 0.9429280397 to 0.9454094293 after expanded training data. The old queue is withdrawn before consumption and the resumed report uses the latter value. The next change adds balanced accuracy 0.8105360802 and dummy balanced accuracy 0.1111111111. Executing the injected defect causes the numerical gate to fail; regenerated analysis restores these checked values and the worker recomputes the current accuracy and macro-F1 gains. Recovery restarts generation from the registered task and current inputs; it need not diagnose the injected defect from its source, and an identical prompt can retrieve cached clean code. Under a manual queue omission, rollback finds the changed decision but leaves the queue: the worker consumes one old job. This separates detecting invalid evidence from actually withdrawing its downstream effects. Both engines show the same behavior under the same interface.

Development failures retained. WF1 was stopped after an audit found verifier answers readable through the actor input mount. Actual use of those answers is not established, but WF1 is excluded from correctness evidence. Its completed logged requests used 51,588 tokens; three saved requests have no response and unknown usage. WF2 fixed isolation and ran all 18 planned initial histories. Only 7 passed: failures included incomplete paired-row handling, response-rate denominators, category strings and result structure. Each recovery arm restored 17 of 54 planned updates: 19 were executed, two recovery analyses failed, and 35 were not executed. WF2 has 97/270 observed normal records and 168/432 observed omission probes. Missing stages remain in planned denominators.

WF3 adds real public-data inspection and fuller tool feedback to the common actor, then reruns all six tasks and all three seeds without replacement. Its 18/18 initial and 54/54 per-arm recovery

results are a new development round, not a corrected substitution into WF2. Improvement between rounds is not attributed to TrustScientist. Both raw rounds and the aborted WF1 are retained. An audit also identified WF2 report prose that incorrectly said calculating means/medians assumes normality, and an omics report that called Pearson correlation unrequested despite the added requirement. These defects illustrate why mechanical report checks do not establish scientific validity. The same ceiling remains in WF3: an omics report describes the correlation as not statistically significant at conventional thresholds, although its saved analysis and required estimates include no significance statistic. This unsupported statement is retained, not silently replaced or scored as scientifically correct.

Measured cost and integration size. Each WF3 recovery method uses 26,855 logical initialization tokens plus 70,006 logical continuation tokens, totaling 96,861. The shared cache makes 90 actual formal requests: 38,268 input and 28,229 output tokens, totaling 66,497 across all arms. Setup adds 1,890 logged tokens. WF2 separately uses 66,063 actual formal tokens and 1,526 setup tokens. These totals are not per-engine savings. Some repeated environment probes lack a cumulative ledger; development and review-assistant usage is unavailable. The interrupted WF1 requests also prevent a complete all-work token total.

The WF3 formal window is 296.75 seconds. Summed direct-tool time is 169.49 seconds with 189.58 CPU seconds; separately logged reference calculations take 7.13 seconds and public-data inspection 1.10 seconds. Parallel tasks overlap, and these sums do not measure latency. The maximum direct-tool RSS across setup and formal execution is 202,344 KiB. GPU-board power samples integrate to 22.73 Wh during the formal window, excluding startup, idle, whole-host energy and review. It cannot be allocated to individual arms. Materialized WF3 files total 66,578,049 bytes, including snapshots and saved outputs; this is neither peak allocated disk space nor bytes written.

The shared adapter has 23 nonblank, non-comment source lines, counting docstrings. The existing TrustScientist store has 204 and the independent generic implementation 68 under that same count; this is not a functionality or maintainability score. The caller names four explicit output paths or one owned root. The adapter contains no task names and is unchanged across both engines and all six tasks; switching the backend changes its constructor. Registration-branch counts are archived, but include test-specific omission logic and must not be interpreted as developer productivity. No external integrator, human development time or cross-project maintenance study is included. The new evidence supports bounded reuse and containment, with no exclusive algorithmic advantage over capable selective replay.

Remaining report evaluation. The archive provides 72 unique review slots for the 18 initial reports and three recovered states per history, with blank human labels. They require checking narrative support, metric and denominator meaning, and justification of the recommended next action. No completed human scientific-accuracy or Agent noninferiority result is claimed. Independent audit contexts use the same model family and provide diagnostic evidence, not external validation.

3.1 EXT1: frozen LangGraph integration on new task families

EXT1 is separate from WF1–WF3. The interface and two existing engines were frozen before task execution and match their WF3 hashes. LangGraph 1.2.16, langgraph-checkpoint-sqlite 3.1.2 and statsmodels 0.15.0 run in a new isolated environment. The original worker and model environments are unchanged. A fresh model context executed the documented environment witness. The

Table 17: EXT1 per-task outcomes for each complete recovery arm. Seeds and backend copies are not independent task families.

Dataset	Initial conformance	Planned updates recovered
co2	0/3	0/9
sunspots	0/3	0/9
nile	3/3	9/9
grunfeld	0/3	0/9
copper	2/3	1/9
macrodata	0/3	0/9

preformal audit independently checked all 12 initial/expanded CSV exports and their analytical references. These are same-family diagnostic checks, not human scientific evaluation or an external developer study.

Data and interventions. The initial input contains the earliest floor(70%) of distinct time points, including all entities at each selected point. Expansion reveals the rest of the historical dataset; it is not newly collected evidence. Base and added metrics are specified before formal requests. CO2 and sunspots use means and intercept-inclusive trends; Nile uses mean and median discharge; Grunfeld uses pooled investment OLS and a within-firm coefficient; copper uses a two-predictor log regression and its R-squared; macrodata uses quarterly log-GDP growth, inflation and growth–unemployment correlation. The six numerical references are recomputed from public data, never from actor estimates. Six separate hand-constructed arithmetic witnesses test the reference implementation. Numerical agreement is not scientific ground truth or an endorsement of these descriptive specifications.

Each seeded history is assigned data expansion, a predefined extra metric and an executed $+0.25$ corruption of one required estimate. Code is generated from current public inputs; the actor cannot read verifier targets. A failure can trigger at most two repairs after the first code attempt. Regeneration can retrieve previously cached clean code and does not prove autonomous diagnosis of the injected bug. The later worker computes a metric count and sum from the queued report, a local reversible diagnostic operation rather than an external scientific intervention.

Fair controls and denominators. TrustScientist, generic replay and a separate native checkpoint-backed baseline each receive the same dependencies, release policy, checkpoint, resource obligations and model responses. Every graph and tool executes; exact-request caching pairs only model responses. The native implementation uses real graph-state receipts and does not import either replay engine. The workload has one active release decision, so it does not test complex multi-decision rollback or crash-transactional equivalence. All 126 initial, 378 normal and 648 omission IDs occur exactly once, including unexecuted slots after an initial failure. Source and task hashes still match the formal freeze. The complete controls tie; no baseline speed or token claim is made from unequal cache warmth.

Concrete recovery and blocking trace. For Nile seed 20261031, expansion changes mean volume from 943.314286 to 919.350000 and median volume from 937.5 to 893.5. Recovery withdraws the old report and pending job before resuming the actual pre-analysis graph checkpoint. The metric update adds a post-1900 minus pre-1900 difference of -235.543953; the report, export and worker use the current estimates. The executed corruption is rejected and regeneration restores the

Table 18: EXT1 omission probes per backend. TrustScientist, generic and native controls agree. Cells show safe withdrawals/executed probes, with 18 planned per cell; parentheses show old jobs actually consumed.

Probe	Explicit paths	Owned root
Omit report	0/5 (0)	5/5 (0)
Omit export	0/5 (0)	5/5 (0)
Omit queue	0/5 (5)	5/5 (0)
Omit follow-up	0/5 (0)	5/5 (0)
Omit data	0/5 (5)	5/5 (0)
Outside-root queue	0/5 (5)	0/5 (5)

checked values. In copper seed 20261032, regeneration fails after data expansion; in seed 20261033 it fails after the added metric. Both stop after withdrawing old effects. Thus 12/12 attempted recoveries contain stale effects, while only 10/12 finish with a valid new continuation. There are 42 unexecuted update slots per arm: 39 after initial failures and three after these later failures. Neither safety nor successful completion is inferred for those slots.

Failures and omissions. The raw failed code, numerical outputs, feedback and all retry costs remain in S4. Failure examples include flat JSON output instead of the required estimates object, omitted regression intercepts and wrong calculations. No task is replaced and no extra attempt is added after formal inspection. Per backend and integration, six probes are planned for all 18 histories, but only 5 valid initial histories support execution. Missing fault observations are unexecuted, not successful safe withdrawals or measured zero old jobs.

Managed probes omit argument-list entries that directory ownership replaces; they are containment tests, not equivalent source-code deletions. A moved queue outside the owned root requires explicit compensation. No natural programmer-error rate, automatic discovery of scientific requirements, concurrent remote consumer safety or irreversible rollback is established.

Complete measured costs and unknowns. Every complete arm uses 58,318 initialization tokens and 27,068 continuation tokens, totaling 85,386 logical tokens including failed analyses and retries. Actual formal model use is 60,673 input plus 20,136 output tokens, or 80,809, across 77 uncached requests. The preformal cache manifest accounts for 6,288 setup/smoke tokens. Formal missing-usage requests: 0. Logical requests include cache hits; physical usage is charged once to the experiment, not credited as a per-backend saving. The fixed token budget conservatively reserves the model's 16,384-token context capacity per in-flight request; unknown usage retains its reservation. Initial setup and formal cache ledgers are separate.

Formal elapsed time is 252.42 seconds. GPU0 board-power samples give an estimated 18.57 Wh over 250.41 seconds, from 125 samples with a maximum gap of 2.02 seconds and 0 recorded sample errors. Trapezoidal integration is an estimate, not calibrated whole-host energy. Model startup, idle time outside the formal window, earlier smoke energy and assistant development/review usage are excluded or unavailable. Persistent model-service CPU and whole-host RAM are not measured. Child-process and direct-tool CPU records are separately supplied; their overlap must not be summed. Materialized bytes are a final inventory, not bytes written or peak disk allocation. No complete financial campaign cost is claimed.

Table 19: EXT1 resource accounting per arm. Tool time is summed; RSS is the largest observed tool process, not aggregate concurrent memory. Stored MiB include this arm's snapshots and quarantine but exclude shared cache and separate initial/fault clones.

Arm	Tool calls	CPU s	Peak MiB	Stored MiB
TrustScientist / owned root	71	23.91	110.0	4.8
Generic / owned root	71	23.89	110.0	4.7
Native LangGraph / owned root	71	23.85	110.0	4.8
TrustScientist / explicit paths	71	23.89	110.0	4.8
Generic / explicit paths	71	23.83	110.0	4.7
Native LangGraph / explicit paths	71	23.86	110.0	4.8
Checkpoint replay only (diagnostic)	71	23.88	109.0	4.8

Integration changes and remaining human work. The unchanged 23-line ownership interface is reused with unchanged 204-line TrustScientist and 68-line generic engines. The new shared framework glue has 26 nonblank/non-comment lines (27 AST statements), and the native baseline 45 lines (64 AST statements); docstrings count as physical lines. A source-derived normal-orchestration excerpt adds nine lines and removes three in one file, with construction, registration and revalidation as its three API touchpoints. The excerpt is a counterfactual source inventory, not a separately authored third-party application or measured developer patch. The framework workflow itself, task code, evaluator, instrumentation and fault harness are listed separately in S4. Line counts depend on style.

The independent-developer packet remains uncompleted. It calls for matched, counterbalanced integration tasks, experience recording, real active/debugging time, missed registrations, support requests, unexpected resource changes and external compensation. Domain review must assess narrative support and next-step justification separately from numerical conformance. No human participant, time, error label or scientific-validity score is fabricated.

Table 20: Responsibilities of the research-agent interface. The mechanisms in the first column are available to a capable generic replay engine; the table makes no exclusive-algorithm or literature-first claim.

Generic mechanism	TrustScientist's explicit application contract	Tested consequence
Dependency invalidation	Version-bound requirements name the task, split, metric and admissible evidence.	Current-value binding and detection of represented missing outputs.
Selective recomputation	A receipt binds the historical action to the policy version, read set and non-evidence context.	Agreement on the earliest changed action.
Checkpoint recovery	A changed action invalidates its linear continuation and registered local outputs.	Re-execution from the preceding checkpoint and quarantine of stale products.
Persistent memoization	Successful unchanged-action attestations survive budget exhaustion but expire on input revision.	Completion across bounded invocations; restart and consecutive-update tests.

4 Exact Recovery Contract and Selective-Replay Boundary

4.1 Software contract relative to selective replay

The software problem is to keep a research agent's evidence, historical commitments and released products mutually consistent as artifacts change. Dependency tracking identifies potentially affected computations; it does not by itself specify the meaning of a research requirement, which action belongs to which policy context, or which products must be withdrawn when that action changes. TrustScientist supplies these application contracts explicitly. Table 20 separates their responsibilities from the generic incremental mechanisms that implement them.

Constructive comparison. A generic implementation can represent each receipt's policy evaluation as a target whose dependencies are its recorded input versions and policy context. An ordered scheduler evaluates dirty targets, compares their actions with the historical actions, and invokes the same rollback adapter at the first change. Persisting each successful version attestation yields the same progress rule. Thus, under the same complete dependencies, stable inputs, deterministic policies and trusted receipt store, this construction implements the CBR interface. The generic-progress control in Section 2.7 instantiates that construction. The V5 generic finite-dispatch control instead recomputes the same registered rule without certificate memoization. The observed ties are therefore expected, and preclude an intrinsic complexity or token advantage over equally capable selective replay. This interpretation is consistent with the separation of scheduling and rebuilding in build systems [2] and with correctness-preserving change propagation [1].

The observable recovery contract. Restrict attention to a quiescent evidence state E and a finite registered linear history $\mathcal{H} = (d_1, \ldots, d_n)$ for which all current policy evaluations $a'_i = \pi_i(E|_{W_i}, x_i)$ are available and total, and the registered resources required for recovery are available and restorable. Missing inputs, unavailable policies and failed recovery remain blocked under Section 1.4; they are outside this completed-response specification. Let $j^*(E, \mathcal{H})$ be the earliest receipt whose deterministic current action differs from its recorded action, or $\perp$ if none differs. The

requested response is

$$\mathcal{R}(E,\mathcal{H}) = \begin{cases} (\text{retain}, \varnothing), & j^* = \bot, \\ (\text{restore}(k_{j^*}), \bigcup_{i=j^*}^{n} O_i), & j^* \neq \bot. \end{cases}$$

The second component identifies exactly the registered outputs to withdraw; the first identifies the checkpoint to restore, or preservation of the current history; retain and restore are distinct response tags. Here k_i is the checkpoint immediately preceding receipt i. Exact preservation concerns identities outside the suffix-output union, including when an identity is registered to both a prefix and a suffix receipt; it does not assert a semantically smallest withdrawal set. This is an exact recovery specification, stronger than the safety policy of blocking every history and withdrawing every output. It remains relative to registered outputs and represented requirements, not arbitrary external side effects or scientific truth.

Information boundary. Let $z = (E, \mathcal{H})$ range over this restricted state domain, and let $P(z)$ encode all information available to a deterministic recovery procedure, including the responses to every permitted query, not merely one chosen execution transcript. The specification $\mathcal{R}$ is total on this restricted state domain. If there exist z_0, z_1 with $P(z_0) = P(z_1)$ but $\mathcal{R}(z_0) \neq \mathcal{R}(z_1)$, no function of $P(z)$ can implement exact recovery on every admissible state.

Proof. A deterministic function receives the same argument on z_0 and z_1, so returns the same response. That response cannot equal two different required responses. At least one state therefore violates the specification. □

This elementary distinguishability argument specifies an interface boundary; it is not a new lower bound for incremental computation. Information may be encoded in another representation or recovered by a query. Omitting a named field alone is insufficient to establish the premise.

Three explicit witnesses explain the boundary. Each restricted interface exposes only a common tuple: the parametric policy schema, the stated old and current numeric summaries, recorded actions, checkpoint identifiers and the global registered-output universe. Hidden selectors, context parameters and ownership assignments are excluded from this tuple as specified below. Every permitted query uses the same deterministic function of this visible tuple and its query arguments in both states; no other state-dependent oracle is available. Equal tuples therefore imply equal query responses, including under adaptive querying by induction on the query sequence. This excludes partial disclosures that reveal the required action without revealing the hidden parameter exactly. These restrictions describe analytic controls, not the equally capable generic comparator. (i) Consider abstract metric-summary records with current accuracy 0.9 against 0.8 for a dummy model, but macro-F1 0.4 against 0.6; historical accuracy 0.9 against 0.8 and macro-F1 0.8 against 0.6 both passed. The comparison selector is hidden. A requirement to exceed dummy accuracy retains PROCEED; one to exceed dummy macro-F1 changes it. Artifact freshness alone cannot choose the intended comparison. (ii) An unchanged policy proceeds when a score exceeds its recorded threshold. An old score of 0.8 supported PROCEED at thresholds 0.5 and 0.7; a current score of 0.6 preserves only the first decision. Removing the frozen threshold from the observable policy context hides the required rollback difference. (iii) In two two-receipt histories, the first action remains unchanged and the second flips. Both have the same registered output universe $\{A, B\}$. Their ownership assignments are respectively $(O_1, O_2) = (\{A\}, \{B\})$ and $(\{B\}, \{A\})$, with the same checkpoints. If ownership is unobservable, the exact withdrawal sets $\{B\}$ and $\{A\}$ differ despite identical changed-input and changed-action information. Withdrawing both remains a possible conservative response, but violates exact preservation.

These are analytic counterexamples to incomplete observations, not additional scientific tasks or performance measurements. A capable selective-replay engine supplied with the complete contract resolves all three. The engineering question is therefore whether a research-agent integration makes these semantics available and enforces the resulting recovery response. The dummy-accuracy repair demonstrates a separate limit: a perfectly implemented recovery interface can maintain an incomplete requirement set. This boundary analysis does not prove that the tuple in Section 1.3 is a unique or globally minimal representation.

Engineering contribution and boundary. The contribution is a reproducible, executable contract joining evidence closure to action-level recovery, with conditional consistency/progress arguments and measured failure cases. Its practical benefit is to make otherwise implicit integration obligations inspectable and testable. The corrected accuracy experiment in Section 2.11 illustrates the boundary: both implementations could faithfully maintain an incomplete requirement set until the missing dummy-accuracy obligation was made explicit. Repairing that shared contract improves completeness for both methods. Neither the dependency representation nor exact replay discovers omitted scientific requirements or establishes unrestricted scientific correctness.

A falsifiable transfer claim. The contract should be judged by whether another replay implementation can consume it and recover the same registered continuation, rather than by a renaming of dependency maintenance. Our generic-progress experiment establishes an internal mechanism control with a shared recovery adapter. It does not establish independent implementation portability, lower integration effort, or an advantage on unseen workflows. Those stronger engineering claims require an independently implemented, equally resumable comparator and held-out workflow integrations with the same contracts, caches and recovery abilities. Such an external validation is not part of the reported results.

Part II
Original Archived Protocols

5 Recorded Case Studies

These four cases were selected after inspecting the saved experiments to illustrate acquisition, version propagation, branch replay, and a binding failure. The complete cohorts remain in the aggregate evaluation. The companion `revision_examples/examples.json` contains the selected records, source selectors, and file hashes. No new model call or human annotation was made for these illustrations. Version and branch updates are controlled interventions; acquisition operates on published artifacts.

5.1 Restoring a reporting state through two evidence actions

Case `1d41a1787bf4` asks for the SVM F1 score on Musk1+. Its source is accessible and intact, but the capsule binding is incorrect and extraction is interrupted. The initial frontier contains binding; extraction cannot proceed until that prerequisite passes.

Step	Executed action	Binding	Extraction
Initial	—	Fail	Unresolved
First	Rebind artifact	Pass	Unresolved
Second	Resume extraction	Pass	Pass

The two actions read the same source, write separate receipts, and change the public evidence digest after each step. Re-audit resolves extraction, quotation, and the semantic rubric. The final binding quotes

```
Classifier svm:  87.5155 1.9971
```

The answer 87.5155 matches the benchmark reference. The guard-free controller closes after two actions costing two declared units. The structured selector also produces a valid closure for this paired case. The guarded ECC record has a correct candidate but declares no closure: historical calibration fails its target. This example connects an executed action to new evidence and re-audit; statistical certification remains unresolved.

5.2 A shared source invalidates several certificates

In capsule `08ec5a4786de2e71`, event 1 changes registered source `f000`. Three reporting chains read that source. Their extraction, binding, and closure nodes are descendants of the same changed input. Figure 4 preserves the recorded dependency structure.

Closure node	Graph cache after re-audit	Stateful LLM cache
`n004`	Fail	Fail
`n007`	Fail	Pass (stale)
`n010`	Fail	Pass (stale)

The stateful Qwen validator rechecks only the first reporting chain and retains the other two closures. Graph and flat dependency tracking both recheck ten affected nodes and retain no stale certificate; full revalidation checks eleven. The unaffected source node keeps its previous value. These counts concern the supplied version contract. They show the effect of executing transitive invalidation in this event; the flat representation provides the same result.

5.3 The first affected decision need not be the rollback point

Capsule `f8ff1f0dc2ec860b`, event 1, changes a source read by both decisions in an authored history. The earlier decision already selected REFINE. The later decision selected PROCEED under the preceding evidence. Counterfactual replay gives:

Decision	Old action	Replayed action	History operation
Earlier	REFINE	REFINE	Retain receipt
Later	PROCEED	REFINE	Invalidate suffix

CBR refreshes the earlier attestation and rolls back before the later decision. The stored rollback index is 1 in the zero-based archive, and the active history retains one decision. Full replay selects the same checkpoint with the same two decision evaluations in this event. The archived certificate-only tracker marks both certificate predicates false but keeps the historical PROCEED action, leaving one obsolete action. The case illustrates why replay compares actions before choosing a checkpoint. It exercises a controlled deterministic routing rule; no replacement scientific experiment is executed.

5.4 A matching value can still bind to the wrong metric

A nominal-clean manifest in `capsule-4180912` asks:

> Report the macro f1 score for the random classifier.

The candidate value is 0.4858800853546077. The saved source context assigns that value to the neural network's *macro recall*. The file exists, the numeric type and path pass, and the observed digest matches the declared digest. The candidate builder also supplies a true relation flag.

TrustScientist accepts those flags and returns CLOSED. The GPT-5.6-Sol response leaves the same manifest OPEN. Its reason identifies both the metric and classifier mismatch. This is a failure of the supplied binding, retained among the nominal-clean examples. The certificate faithfully records the represented checks, while its accepted relation flag leaves the claim unsupported by the quoted context. The example explains why nominal clean closure is reported separately from established semantic correctness.

6 Metrics and Repeated Decisions

Let A_i indicate a closed prediction and let y_i and $\hat{y}_i$ be the gold and predicted relations. The relation vocabulary is SUPPORTS, REFUTES, and no evidence (NEI). Natural language inference (NLI) classifies this text–claim relation; a large language model (LLM) supplies an additional judge. NEI-FCR denotes the no-evidence false-closure rate. With $N = 339$, $N_{\text{NEI}} = 130$, and $N_{\text{evidence}} = 209$:

$$\text{coverage} = N^{-1} \sum_i A_i,$$

$$\text{NEI-FCR} = N_{\text{NEI}}^{-1} \sum_i A_i \mathbf{1}[y_i = \text{NEI}],$$

$$\text{valid recall} = N_{\text{evidence}}^{-1} \sum_i \mathbf{1}[y_i \neq \text{NEI}, \hat{y}_i = y_i],$$

$$\text{selective risk} = \frac{\sum_i A_i \mathbf{1}[\hat{y}_i \neq y_i]}{\sum_i A_i}.$$

Table 21: Post hoc diagnostic on evidence-bearing pairs only.

Rationale	Pairs	Closed	Correct	Wrong
Hit	118	79	78	1
Miss	91	28	20	8

Macro-F1 averages class F1 after mapping unresolved predictions to NEI. No claim-level artifact certificate is inferred from a support or refutation label. The exact frozen NLI checkpoint is:

```
MoritzLaurer/
DeBERTa-v3-base-mnli-fever-anli
```

The SciFact LLM endpoint is `deepseek-chat`, distinct from the CORE manifest baseline. The training/development unit is a claim–document pair; pairs sharing a claim form a dependence cluster. The current bootstrap analysis resamples 300 claim clusters with 5,000 draws. It differs from an older row-bootstrap script retained in the project.

All five saved SciFact runs produce identical operating-point labels for the LLM judge, global consensus, and TrustScientist policies. The sample variance of coverage, NEI-FCR, valid-claim recall, Macro-F1, and selective risk is zero. NLI-only, risk-controlled NLI, and Mondrian policies are deterministic in this evaluation. Repetition still changes some LLM confidence values and therefore their rankings. It does not provide five independent test sets.

The 20 TrustScientist errors among 118 selections decompose into 11 NEI closures and nine reversed SUPPORTS/REFUTES relations. The remaining 98 are correct. Of the 209 evidence-bearing pairs, 111 are not correctly resolved: 102 remain unresolved and nine are assigned the wrong relation. This accounts for both the selective-risk and valid-recall denominators.

6.1 Rationale-hit diagnostic

The frozen selector hits a gold rationale on 118/209 evidence-bearing pairs (56.5%). Table 21 stratifies the saved TrustScientist predictions by that indicator. Rationale misses account for eight of nine wrong selected relations on evidence-bearing pairs. Correct decisions remain possible on some misses because a non-annotated sentence may still convey relevant information. This post hoc stratification does not replace an oracle-retrieval intervention and cannot causally assign each error to a retriever or verifier. NEI pairs have no gold rationale and are excluded.

6.2 The original matched-coverage analysis

At each point, the saved procedure retains the top k natural non-NEI closures for each policy. The saved shared interval selects between 17 and 93 of the 339 pairs, using the resulting coverage coordinates. The ordinate is NEI-FCR, not selective risk. The reported area is the trapezoidal integral divided by interval length; it therefore has the scale of a mean NEI-FCR over the interval.

The typed ranking uses $\min(1, s/\tau)$, so every accepted typed-NLI example has score one. Stable sorting leaves these ties in dataset order. On their accepted subsets, the full and global consensus rankings reduce to LLM confidence because the NLI priority is also clipped to one. The resulting areas are reproducible but sensitive to this ranking design; they do not establish superiority over a well-ranked comparator.

Table 22: Original normalized NEI-FCR integrals. Variance is the sample variance across five saved runs, not a confidence interval.

Policy	Mean	Variance
Direct NLI	0.05040	0
LLM judge	0.06580	0.00001775
Typed NLI	0.04787	0
Global consensus	0.02597	0.000004664
TrustScientist	0.02413	0.000006645

7 CORE Manifest Protocol and Baselines

The 79 unique question targets come from all 45 CORE-Bench test capsules. The evaluator searches extracted text for the known reference value and accepts the first match. It assigns the target question to the candidate record and sets its clean relation flag to true. Eighteen of 62 extracted candidates would fail the preceding weak lexical relation heuristic. That heuristic itself has no independent accuracy validation, so 18 is a code-path sensitivity count, not a count of proven wrong bindings. The 62/79 endpoint is reference-assisted candidate retrieval. Seventeen nonextractable targets remain OPEN with action `enable_or_extend_artifact_parser`.

Both LLMs see the same 372 manifest inputs and text prompt. Reconstructing inputs from the decision table reproduces the saved input JSONL exactly. The prompt asks the model to close only if an admissible artifact satisfies every mandatory condition, to return OPEN for missing/invalid conditions, and UNVERIFIABLE for evidence it cannot inspect reliably. It asks for a JSON state, confidence, failed condition, and reason. The user message includes claim, requirement, type, mandatory flag, artifact manifest, and semantic evidence. It does not request four separate type decisions with a shared external aggregator.

The manifest supplies validity flags for artifact type, path, digest, and observed relation. The type is SUFF for nominal clean inputs, AUTH for missing-artifact, wrong-type, invalid-path, and stale-digest controls, and REL for wrong-question controls. It consequently changes with the control condition. Gold states, condition labels, and final TrustScientist decisions are retained for analysis but excluded from the model-facing prompt.

DeepSeek requests `deepseek-v4-flash` with temperature zero and JSON object output. GPT requests and returns `gpt-5.6-sol`, with reasoning effort `none` and a strict JSON response schema. The text prompt and evidence inputs are matched; the model and output protocol differ. No new model calls were made for the present reanalysis.

For DeepSeek, 55 clean-closed cases generate 275 corresponding corruptions. Of those, 148 remain closed (53.8%); another corrupted closure comes from a clean-open case. This explains why conditional preservation is 148/275 while residual corrupted closure is 149/310. GPT has 0/295 conditional preserved closures. The main paper uses the unconditional corrupted denominator so all policies are compared on the same states.

The GPT clean nonclosures expose three mismatches in the saved contexts:

- `capsule-4180912`: the question requests random-classifier macro-F1; the candidate value instead describes the neural network's macro recall.
- `capsule-8234136`: the question requests lowest average wait time, while the candidate appears in a CPU-utilization plot.

Table 23: CLOSED decisions by supplied manifest condition. Nominal clean states have not been independently validated for semantic correctness.

Condition	DeepSeek	GPT	TrustScientist
Nominal clean	55/62	59/62	62/62
Missing artifact	0/62	0/62	0/62
Wrong type	39/62	0/62	0/62
Invalid path	49/62	0/62	0/62
Stale digest	29/62	0/62	0/62
Wrong question	32/62	0/62	0/62
All corrupted	149/310	0/310	0/310

- `capsule-9832712`: the question requests a Figure 2 availability percentage, while the number 100 comes from a bibliography title.

These are evidence of limitations in the candidate-building procedure. Counting all nominal clean closures as correct would reward acceptance of such mismatches.

7.1 Cumulative gates and restoration

The archived cumulative value, type, artifact-binding, and relation stages leave 248, 186, 62, and 0 of the 310 controls closed. These exact multiples follow from five equally sized classes: value checks remove missing-artifact controls, type checks remove wrong-type controls, path/digest checks remove two further classes, and relation checks remove wrong-question controls. This is coverage of the constructed test conditions rather than an effect size on a natural-error distribution.

The action name corresponds to the first failed gate. Restoration calls the same verdict function on the saved nominal clean record; all 310 return CLOSED. No repair command runs in this test. The main trace uses clean row 1 and stale-control row 5 of the saved decision table (one-based data records). The reconstructed certificate contains the claim/requirement ID, artifact path, value, expected and observed digest, passed predicates, and decision. Its provenance record identifies it as reconstructed from the archive.

8 Fresh-Workspace Diagnostic

Each run executes the same 20 public-train capsules in a fresh workspace. Strict success requires a locally produced result within the declared target. The saved run also attempts one official-artifact recovery; that recovery is excluded from strict task success. The 58/59 question count refers to initial fresh-output matches, before recovery. Compatibility fixes were prepared for this host, so the diagnostic does not measure autonomous repair selection.

Table 24: Recomputed fresh-execution outcomes and summed task wall times.

Run	Strict tasks	Fresh questions	Seconds
1	19/20	58/59	840.6
2	19/20	58/59	745.9
3	19/20	58/59	741.5
4	19/20	58/59	747.7
5	19/20	58/59	754.0
Mean	19/20	58/59	765.9

The retained failure is `capsule-4884085`: its archived CUDA 10.1 binary is incompatible with the Ampere GPU, and CPU fallback misses the reference target. It remains in every denominator. All 20 tasks' decisions and 59 question transitions were compared with the aggregate summaries.

9 Exploratory Natural-Claim Preannotation

An archived pilot contains 100 saved claims, 25 from each of four agent-generated papers with available source bundles. A generic artifact adapter infers required evidence from claim text/type and checks available artifact categories. Its saved output differs from the CORE policy operating on supplied predicate flags; the two experiments must not be pooled.

GPT-5.6-Sol preannotations were saved separately for each paper, withholding previous judgment values. Recounting the per-claim JSONL files gives 100 adapter CLOSED decisions versus four model CLOSED, 51 OPEN, and 45 UNVERIFIABLE decisions. The saved comparison CSV agrees with every record, and the source and input hashes match the archived snapshot. Model closure counts by paper are 0, 3, 0, and 1 (25 claims each); these are descriptive counts without a correctness reference or a population estimate.

The sample favors introductory and related-work claims; cited external sources are often outside the local bundles. Inputs retain paper identities and provenance cues, so withholding previous judgments does not establish independent blinding or matched evidence access. Each of the two human annotation packets contains 100 records but no populated eligibility, requirement, or claim-state fields. The adjudication table also has no completed human states.

Neither model output nor adapter output is gold. We report no requirement recall or natural false-closure rate from this pilot. An independent evaluation needs a frozen evidence boundary, eligible claim units, and adjudicated requirement identities before one-to-one scoring.

10 Evidence Archive and Assistance Disclosure

The reanalysis checks tables, reconstructs inputs, compares predictions, replays predicates and restoration, and recomputes SciFact and execution metrics. Derived JSON records input hashes using SHA-256. This verifies saved records without reopening original capsule binaries or establishing the correctness of supplied relation flags. The exact manifest-auditor prompt is archived separately.

AI assistance through Codex was used on 25–30 September 2026 to revise the manuscript, inspect code and saved outputs, verify literature metadata, and write the offline reanalysis. The original figures were retained. The session identified its model family as GPT-6; a precise backend revision was not available. The author's revision brief requested an AAMAS research-agent framing, omission risk, consistent semantic metrics, scoped corruption and action claims, updated related work, raw-output verification, and a worked trace. This September 30 request asked to update the paper, assess effectiveness, and run additional experiments and revise Overleaf without new human annotation. Codex implemented the paired semantic-gate design, independent scoring code, and manuscript revisions under that brief. A subsequent request to test stateful graphs against carefully prompted validators led to the dynamic file-version experiment, its dependency-table control, and independent replay scorer; that protocol and the exact model prompt are archived. The new experiment uses actual model calls; the separate calibration construction uses stated assumptions. A further request led to the frozen discriminating-acquisition pilot and a separately labelled post hoc choice-interface diagnostic on newly acquired public-train capsules; their prompts, source hashes and model receipts are archived. Experimental model runs and their saved prompts are documented separately; AI-generated labels are not described as human or expert validation.

11 Archived Restoration Trace

11.1 Worked trace: a stale artifact manifest

The archived control for `capsule-5507257` follows one requirement through the interface. Its task asks for the multitask model's final test accuracy. Reference-assisted extraction locates 96.12499 (rounded) in `results/output`. The represented checks require presence, an accepted type, a permitted path, matching digest, and the declared question relation. These checks cover this benchmark target; the trace does not create replication or comparison requirements absent from the target.

The stale control retains the value and context but replaces the observed digest with 64 zeroes. The AUTH predicate comparing observed and expected digests fails, leaving the requirement OPEN. The saved action specification is `refresh_stale_artifact`, linked to the originating requirement and its digest test. For a real executor, the expected output would be a refreshed artifact record whose digest can be checked again.

In the archived experiment, the intervention restores the known clean manifest. Its digest again matches the expected value beginning `bf0c92e1`. Re-audit repeats the same checks and returns CLOSED. A reconstructed certificate view contains the requirement identifier ending `4f0c98b26e`, the artifact reference, extracted value, digest comparison, predicate outcomes, and final state. The underlying records support this view, but no originally issued certificate file is archived. The sequence shows claim → requirement → candidate binding → failed predicate → action specification → changed manifest → re-audit → certificate view. The actual intervention is manifest restoration, not execution of the proposed refresh action.

Table 25: Earlier controlled protocol, 144 attempts per policy. Error incidence uses all attempts; conditional risk uses closed attempts.

Policy	Errors/all	Errors/closed	Valid/all
Direct	98/144	98/121	23/144
ReAct	97/144	97/120	23/144
ReAct + Reflection	97/144	97/120	23/144
LLM Judge	97/144	97/120	23/144
TrustScientist	1/144	1/24	23/144
TrustScientist + loop	4/144	4/96	92/144

12 Controller Protocol and Historical Evidence

The control experiment freezes the controller, adapter, case list, calibration split, and source hashes before full evaluation. Each case is evaluated under six policies and budgets 1, 2, and 4. The local Qwen3-8B endpoint uses temperature zero, seed 20260929, non-thinking JSON mode, and at most 512 output tokens per request. Identical request payloads share cached responses across policies and budgets. Rollouts are policy executions on shared model outputs, not independent model replications. Tool executions still produce separate receipts. Capsule-cluster percentile intervals use 5,000 bootstrap resamples; degenerate zero-error intervals do not establish a population risk bound.

Inference uses vLLM 0.19.1, bfloat16, and a 16,384-token context window. The Qwen3-8B weight snapshot is `b968826d9c46dd6066d109eabc6255188de91218`. At budget 4, mean acquisition cost over all natural attempts is 1.83 units for ECC and 1.70 for the structured selector; on controlled attempts it is 1.67 and 1.89, respectively. Costs of unsuccessful attempts are included. The corresponding mean charged token counts are 10,840 versus 13,385 on natural tasks and 1,959 versus 2,112 on controlled states. They include shared cached requests; actual unique inference consumes 818,472 tokens.

Graph dependencies are access $\rightarrow$ integrity and binding; integrity and binding $\rightarrow$ extraction; extraction $\rightarrow$ quotation $\rightarrow$ semantic adequacy. The cost-aware score counts failed nodes in a fixed tool-effect map and divides by declared cost. Refresh and fetch can affect four structural nodes; rebind affects binding, resume affects extraction, and search affects quotation and semantic adequacy. The graph-free order is resume, rebind, search, fetch, refresh. The budget-selection ablation uses refresh, fetch, search, rebind, resume. Exact model prompts, decisions, action receipts, and evidence hashes are retained with the experiment release. This is an authored scheduling heuristic, not a learned value-of-information estimator.

12.1 Earlier formal artifact protocol

We preserve the earlier 1,371-record protocol separately: 270 requirement records, 864 controlled attempts, and 237 natural-case model runs. Its 123 natural failure trajectories include abstentions and protocol failures; they are not 123 independently annotated false closures. The controlled cohort has 24 nominal-clean bundles from 11 capsules and five corruption variants each. It uses the same local model family but a different policy, input and scoring protocol from the new graph experiment.

The earlier headline percentages 68.06%, 67.36%, 0.69%, and 2.78% describe *error incidence*, not risk conditional on closure. Both recovery rounds recover 69/120 corrupted states (57.50%); the

second round adds no successes. False recovery is 3/72 declared recoveries (4.17%). The executor restores published source artifacts; these are not autonomous scientific repairs.

In a post hoc replay, applying the same deterministic checks to the saved Direct, ReAct, Reflection, and Judge reports yields 24 closures, one error, and 23 correct closures for each method. All 144 decisions agree with the static TrustScientist policy. These 576 replay records use no new model calls and show that the earlier gap cannot be attributed to typing alone.

12.2 Additional fresh execution diagnostic

A separate new execution of the same 20-task, 59-question subset initially reproduces 12/20 tasks and 40/59 questions. Operator-selected environment repairs and six reruns increase this to 18/20 and 56/59. Extending the output parser for scalar wrappers, without changing the raw scientific output, produces 19/20 and 58/59. This sequence is retained separately from the five archived runs and the autonomous artifact-acquisition loop. It does not show autonomous selection of environment repairs. The remaining reference mismatch stays in the denominator.

Table 26: All control settings. V and F are valid and false final closures; Rec is valid recovery among initially unresolved states. A and K are mean actions and declared cost among valid final closures; a dash means undefined. Candidate outcomes are shown in the main text. Mean tokens charge shared cached calls to each policy, and do not count independent inference.

Cohort	Policy	Budget	V/all	F/all	Rec	A	K	Mean tokens
Natural	ECC	1	0/46	0/46	0/24	–	–	5483
Natural	ECC	2	0/46	0/46	0/24	–	–	8752
Natural	ECC	4	0/46	0/46	0/24	–	–	10840
Natural	w/o graph	1	0/46	0/46	0/24	–	–	5483
Natural	w/o graph	2	0/46	0/46	0/24	–	–	8752
Natural	w/o graph	4	0/46	0/46	0/24	–	–	10840
Natural	w/o risk guard	1	18/46	4/46	0/24	0.00	0.00	5483
Natural	w/o risk guard	2	21/46	7/46	3/24	0.14	0.29	8752
Natural	w/o risk guard	4	21/46	9/46	3/24	0.14	0.29	10840
Natural	w/o budget selection	1	0/46	0/46	0/24	–	–	5483
Natural	w/o budget selection	2	0/46	0/46	0/24	–	–	8752
Natural	w/o budget selection	4	0/46	0/46	0/24	–	–	10840
Natural	w/o re-audit	1	0/46	0/46	0/24	–	–	5483
Natural	w/o re-audit	2	0/46	0/46	0/24	–	–	5483
Natural	w/o re-audit	4	0/46	0/46	0/24	–	–	5483
Natural	Structured LLM	1	18/46	4/46	0/24	0.00	0.00	5605
Natural	Structured LLM	2	21/46	7/46	3/24	0.14	0.29	10245
Natural	Structured LLM	4	21/46	9/46	3/24	0.14	0.29	13385
Controlled	ECC	1	0/36	0/36	0/24	–	–	654
Controlled	ECC	2	0/36	0/36	0/24	–	–	1303
Controlled	ECC	4	0/36	0/36	0/24	–	–	1959
Controlled	w/o graph	1	0/36	0/36	0/24	–	–	654
Controlled	w/o graph	2	0/36	0/36	0/24	–	–	1303
Controlled	w/o graph	4	0/36	0/36	0/24	–	–	1303
Controlled	w/o risk guard	1	0/36	0/36	0/24	–	–	654
Controlled	w/o risk guard	2	11/36	1/36	11/24	2.00	2.00	1303
Controlled	w/o risk guard	4	22/36	2/36	22/24	1.50	2.50	1959
Controlled	w/o budget selection	1	0/36	0/36	0/24	–	–	654
Controlled	w/o budget selection	2	0/36	0/36	0/24	–	–	654
Controlled	w/o budget selection	4	0/36	0/36	0/24	–	–	1967
Controlled	w/o re-audit	1	0/36	0/36	0/24	–	–	654
Controlled	w/o re-audit	2	0/36	0/36	0/24	–	–	654
Controlled	w/o re-audit	4	0/36	0/36	0/24	–	–	654
Controlled	Structured LLM	1	0/36	0/36	0/24	–	–	880
Controlled	Structured LLM	2	10/36	1/36	10/24	2.00	2.00	1463
Controlled	Structured LLM	4	22/36	2/36	22/24	1.18	2.82	2112

13 Operational Labels and Policy Certification

13.1 What the existing labels establish

This study adds no human-annotation experiment. CORE questions and reference answers supply an existing external task specification. In the control run, the final evaluator conjuncts the same structural and literal-binding checks used by the controller with reference-answer matching. Withholding references from action selection prevents direct answer access; it does not make the shared checking implementation an independent validator. The reported counts retain this frozen operational definition.

A retrospective paired audit matches task identifiers across the unguarded controller and structured selector at budget 4. All 46 natural and 36 controlled cases agree on both the closure decision and operational validity. The audit does not assert identical generated text or identical action sequences.

Source fidelity and reference agreement can diverge. For a question about unavailable analysis scripts, a saved report quotes a row for processed data. For a sample-size question, another quotes a regression-estimate row. These are observable category mismatches. A separate classification report quotes an output value that differs from the benchmark reference executions. That disagreement alone cannot establish a wrong quotation or distinguish execution variation from a context error. Plot-based answers can also lose spatial relations when figures are flattened into text. These examples motivate separate scoring of task-answer agreement, source-report fidelity, and execution or artifact-contract validity. The existing aggregate labels have not been retrospectively replaced with inferred semantic gold.

13.2 A sufficient episode-level calibration construction

The following construction specifies a route to certification; it is not a guarantee attained by the historical guard. Fix a finite family of M complete episode policies before calibration, including their prompts, tools, budgets, internal stopping rules, and model randomization distributions. Each policy may acquire evidence adaptively within an episode. Fix an operational binary error evaluator and a calibration sample size before observing its outcomes. Calibration episodes must be independent and identically distributed from the deployment population and unused in policy or evaluator development. Episodes must not share adaptive memory. A fixed benchmark collection alone does not establish this population-sampling assumption.

For policy j, let C_j indicate final closure and L_j indicate evaluator error. Its target is $r_j = \Pr(L_j = 1 \mid C_j = 1)$, for positive closure probability. Among the calibration episodes let N_j close, with K_j errors. For a fixed policy, conditional on N_j, the accepted-error count is binomial with parameter r_j. Define the one-sided Clopper–Pearson upper bound

$$U_j = \begin{cases} 1, & N_j = 0 \text{ or } K_j = N_j, \\ Q_{1-\delta/M}(\mathrm{Beta}(K_j + 1, N_j - K_j)), & \text{otherwise}, \end{cases} \tag{3}$$

where Q_p denotes a distribution's p-quantile. Certify only policies with $N_j > 0$ and $U_j \le \alpha$, for a prespecified tolerance $0 < \alpha < 1$. With probability at least $1 - \delta$ over calibration,

$$\forall j \text{ certified}, \qquad r_j \le \alpha. \tag{4}$$

To see this, the exact binomial bound has undercoverage probability at most δ/M for each fixed policy, also after averaging over N_j. A union bound over the M policies gives simultaneous coverage.

Any selection among the certified policies preserves this statement. The policies' outcomes may be correlated on shared episodes; independence is required across sampled episodes, not across policies. This is a standard finite-family calibration argument, consistent with Learn then Test, not a new risk-control theorem.

Internal adaptive acquisition is part of the frozen policy's output map. The construction neither bounds risk conditional on every observed history nor permits changing the policy after calibration. It does not allow repeatedly inspecting results and stopping calibration at the first favorable bound. If several questions share a capsule, one can prespecify one question per independent capsule, or define a capsule-level any-error loss; the latter certifies a different, capsule-level target. Replicate seeds and controlled mutations are not independent capsules. The guarantee concerns the chosen operational error, and does not repair erroneous labels or omitted obligations.

Even with zero accepted errors, the bound is $U_j = 1 - (\delta/M)^{1/N_j}$. At $\delta = 0.05$, a fixed policy needs at least 59 accepted independent units to certify $\alpha = 0.05$, or 29 to certify $\alpha = 0.10$. A family of ten policies needs 104 or 51, respectively. These are mathematical best-case requirements, not achieved sample counts. Nonzero errors require more evidence. The current historical calibration and previously inspected evaluation cases do not provide this new policy-matched calibration sample. No policy is certified by this construction in our results.

14 Paired Semantic-Gate Protocol

The acquisition failures motivate a narrower test: once a report and its evidence are fixed, does checking the requested scope improve the closure decision beyond structural checks and answer agreement?

14.1 Paired design and independent scoring

We use all 79 existing natural reporting questions. These questions were available during development; this is an exploratory comparison, not an untouched test set. A question-only retrieval preserves table rows and local headings. Each question receives one Qwen3-8B report, one structured rubric review, and one blind second answer with scope witnesses. The second answer does not see the proposed report or reference answer. All decision rules reuse these same outputs, making the comparison paired at the report level.

The common contract checks artifact structure, literal answer support, the represented question, and declared completion. A structured-rubric gate adds the six semantic checks used in the acquisition study. An agreement gate instead requires agreement with the second answer, grounded candidate values, and no declared ambiguity. The scope gate adds literal question anchors and evidence quotations for the requested dimensions, including metric and figure identifiers when present. Comparing the latter two removes only the scope conditions; retrieval, prompts, answers, and all other checks stay fixed. The gate checks a fixed metric vocabulary and figure markers; other dimensions depend on the model's proposed inventory. Literal witnesses expose a relation for inspection but establish neither its truth nor inventory completeness.

A separately implemented evaluator scores agreement with the benchmark reference values, without calling the controller's audit or binding functions. It independently checks source-file digests and literal answer support as separate diagnostics. Semantic source fidelity remains unlabelled. Correct and wrong closures below therefore refer to reference agreement; they do not certify unrestricted scientific validity. Unresolved and malformed replies remain in the denominator. Capsule resampling gives paired uncertainty intervals, preserving within-capsule dependence.

14.2 Execution and uncertainty

The experiment uses 45 capsules and 237 actual model requests, with no transport errors. Six replies fail JSON parsing and remain unresolved. The saved responses account for 1,044,094 tokens across the three requests per question. Temperature is zero, the seed is 20270928, the maximum output is 1,536 tokens, and the server context limit is 32,768 tokens. The Qwen3-8B checkpoint is the same as in the control study. Report generation and the blind witness receive identical public evidence bundles; the rubric review also receives the proposed report. No model receives benchmark reference values or the private task identifiers.

Each gate can be evaluated from the shared report and at most one additional model output. The three-call study cost is not the deployment cost of each gate, and no cost advantage is inferred from this replay design. There are no acquisition rounds after the initial retrieval; this comparison isolates a termination decision rather than an autonomous recovery policy.

The separately implemented evaluator treats a numeric answer as matching when it lies within the benchmark reference range, extended by the maximum of 10^{-5}, 5% of the range width, and 0.1% of the absolute lower endpoint. Text answers use normalized exact matching. This preserves the existing operational scoring tolerance without calling its implementation. Reference agreement and literal support do not establish correct row/column semantics. All 79 source-integrity checks

Table 27: Paired gates on 79 natural questions. Correct/wrong refer to benchmark answers. Risk is wrong/closed; all methods share generated outputs.

Gate	Correct	Wrong	Risk (%)
Literal contract	44	11	20.0
Structured rubric	35	4	10.3
Answer agreement	34	6	15.0
Agreement + scope	25	5	16.7

pass; 55 reports have literal answer support. Semantic fidelity remains unknown for all questions under independent labels.

We resample the 45 capsules with replacement 10,000 times and compute each rate after pooling the sampled questions. The same draws are used for paired differences. Scope witnesses minus answer agreement yield a successful-closure difference of -11.4 percentage points (95% interval $[-19.8, -2.9]$) and a false-termination difference of -1.3 points ($[-4.2, 0.0]$). Relative to the structured rubric, the corresponding differences are -12.7 points ($[-23.7, -2.9]$) and $+1.3$ points ($[-4.2, 6.9]$). These exploratory intervals describe the observed capsule collection; they do not repair prior use of the tasks during development or provide a deployment risk certificate.

Validation checks the frozen experiment hash, completeness and uniqueness of cases, three requests per question, public-input equality, absence of reference fields and private identifiers in requests, and nesting of scope acceptance within agreement acceptance. Evaluator regression cases include a valid source, a wrong answer, altered extracted text, and a changed source file. The code, protocol, per-question gate decisions, and full responses are retained.

14.3 Complete paired-gate results

We isolate the remaining semantic decision using all 79 existing natural questions from 45 capsules. Each question receives one context-preserving, question-only retrieval, a Qwen3-8B report, a structured rubric review, and a blind second answer with scope witnesses: 237 actual model requests. The questions were used during development, so this is an exploratory study. All gates reuse the same evidence and model outputs.

The common contract checks structure, literal answer support, the represented question, and declared completion. The rubric adds the same six semantic checks used by the controller. Answer agreement instead adds a blind second answer, grounded candidate values, and no declared ambiguity. Scope witnesses add literal question anchors and evidence quotations for requested dimensions, with mandatory metric and figure checks. The agreement–scope contrast removes only those scope conditions; it is a paired component ablation. Scope completeness and the truth of a quoted relation are not established by these syntactic checks.

A separate evaluator scores benchmark-reference agreement without invoking controller functions. Source-file integrity and literal support are checked independently and kept separate from the primary answer score. Thus a wrong closure below means reference disagreement, not necessarily an unfaithful quotation. Malformed replies remain unresolved in the denominator; there is no independent semantic annotation. Full protocol and scoring tolerances appear in the appendix.

Adding scope witnesses rejects one reference error but loses nine correct closures. The paired capsule-bootstrap difference in successful closure is -11.4 percentage points (95% interval $[-19.8, -2.9]$); the difference in false termination is -1.3 points ($[-4.2, 0.0]$). Selective error increases from 15.0% to 16.7%. The structured rubric has the strongest observed balance among

the semantic gates, with 35 correct closures and four errors. The scope extension is therefore not adopted as an improvement to the controller. This result locates a limit of inspectable certificates: requiring more literal witnesses can discard valid answers without resolving semantic misbinding. The experiment tests a gate, not an action-selection advantage.

15 Dynamic Version-Contract Protocol

15.1 Inputs, checks, and fixed event sequence

The inputs are the saved public evidence bundles and reporting outputs of the 79-question paired-gate study. We copy 124 source files and generate separate receipt and report records, preserving 45 capsule boundaries. Identical source paths share one source node within a capsule. Each question adds extraction, binding, and closure nodes; there are 361 nodes in total. A check passes when its registered file digests and parent checks pass. Closure additionally requires the archived initial literal-contract decision. The 55 initially passing contracts are not independently verified scientific answers.

The fixed sequence contains nine events: a change to an unrelated file; changes and restorations of a source, receipt, and report; and a simultaneous source and receipt change followed by restoration of both files. Targets follow lexical order, without selecting for maximum sharing. Updates append a fixed marker to isolated copies; restoration copies the original bytes back. Receipt and report files are checked against registered digests, so these updates test version mismatch, not successful parsing of corrupted JSON. The source and receipt in the multi-input event are distinct files.

All policies share a topological executor and a budget equal to the capsule's node count per event. Selected nodes are first invalidated, then checked in topological order. A failed parent propagates failure without reading child files. Every selected predicate evaluation counts as a check, including such short circuits. The graph index is constructed once per capsule. The flat implementation seeds direct input matches and scans parent lists to a fixed point. Neither policy asks an LLM to determine the changed-file dependency closure. Full re-audit also skips changes outside the registered input set.

The independent evaluator imports no graph or checker code. It reconstructs every event's bytes from immutable originals, verifies the recorded digests, and evaluates direct certificate-to-file contracts. Original source files are checked for byte identity after the experiment. A smoke trajectory on the largest graph passes these checks before the formal study. Its nine model requests are separate from the 405 formal requests; no policy adjustment is made after the smoke run.

15.2 Prompted stateful comparator

Qwen3-8B uses the existing checkpoint, temperature zero, seed 20260930, and a 768-token output cap. Each request includes checks in topological order, their direct files and parent identities, the changed-file identifiers, persistent predicate cache, full previous event history, and common budget. Event types, mutation/restore labels, reference answers, and oracle validity are withheld. All 405 formal responses produce valid plans; no fallback is invoked. They consume 232,958 actual tokens. This is an invalidation-planning test; it does not ask the model to reinterpret scientific artifacts.

The system prompt directs the model to start with every check whose direct inputs changed, repeatedly add all dependents until reaching a fixed point, include intermediate and closure nodes even if already false, propagate restorations, skip unregistered files, and return only an invalidation list. Malformed output conservatively selects full revalidation. The complete prompt is retained in the frozen protocol. This gives the comparator the same dependency information and the intended algorithm, not a generic "check the answer" instruction. Tool-mediated execution of that algorithm is represented by the flat-table comparator.

Table 28: Dynamic version-contract tests. Stale: stale certificates accepted; valid: valid certificates retained. Checks exclude common initialization.

Policy	Stale / 163	Valid / 332	Checks
Full re-audit	0	332	2,888
Stateful LLM	38	327	1,624
Dependency graph	0	332	1,530
Flat dependency table	0	332	1,530
No re-audit	163	332	0

15.3 Denominators, uncertainty, and costs

The 79 questions over nine events yield 711 certificate decisions per method. Of these, 216 belong to initially non-passing contracts. The remaining 495 comprise 332 valid and 163 stale contract instances. Counting all invalid instances as the stale denominator would dilute the prompted validator's error rate from 38/163 to 38/379. We use the former. The valid-retention count includes preserved and restored certificates; it is not recovery from a scientifically wrong answer.

We resample capsules 10,000 times, retaining all events and questions within each sampled capsule. The LLM stale-acceptance interval is $[12.8, 35.2]\%$; its valid-retention interval is $[96.2, 100.0]\%$. The graph's check reduction relative to the LLM is 5.8% with interval $[-7.4, 13.8]\%$, so no check-cost advantage over that selector is established. Zero observed errors give a degenerate empirical bootstrap interval; this is not a population risk certificate.

Full re-audit reads 1,915 files and 244,850,308 bytes across the formal events; graph and flat tracking each read 709 files and 70,096,968 bytes. The LLM reads 862 files and 43,905,679 bytes, partly because it misses required revalidation. Its smaller byte count therefore cannot be read as safe efficiency. Common initialization is reported separately in the raw records.

Accumulated checker time is 0.500 seconds for full re-audit, 0.157 for the graph, and 0.156 for the flat table. Graph index construction adds 0.000564 seconds. The LLM's accumulated request time is 297.184 seconds across concurrent trajectories. These sums are not job elapsed time. The shared server uses a warm file cache and randomized policy execution order; the small deterministic timings are diagnostics, not evidence of a general wall-clock speedup.

15.4 Executed shared-source failure

Figure 4 shows event 1 in capsule `08ec5a4786de2e71`, which updates source `f000`. Ten checks require revalidation: `n000` and `n002` through `n010`. The prompted validator selects only `n000`, `n002`, `n003`, and `n004`. It follows one downstream chain while omitting two others; cached closures `n007` and `n010` remain true despite stale source versions. Both deterministic trackers invalidate all ten checks and match the full oracle. Across events, the LLM has 227 exact invalidation sets, 93 sets missing required checks, and 85 with extra checks. The latter two categories need not be treated as disjoint by definition. Seventeen stale acceptances occur on single-source updates and 21 on simultaneous source–receipt updates. This trace demonstrates missing propagation; it does not establish an error rate for naturally occurring artifact changes.

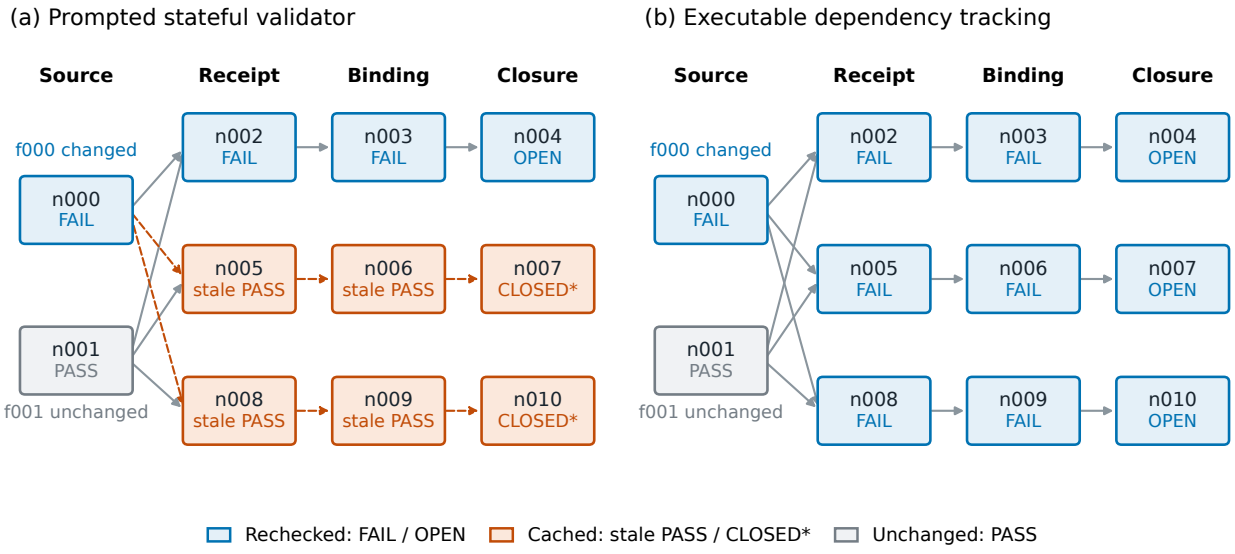


Figure 4: An executed shared-source update (capsule `08ec5a4786de2e71`, event 1). Both policies receive the same dependencies and changed-file ID. The prompted validator rechecks one of three affected reporting chains, leaving two cached closures stale (`CLOSED*`). Executable dependency tracking rechecks all affected nodes. The flat-table implementation produces the same right-hand result. Edges and states come from the recorded run; failure means a version-contract mismatch, not a scientifically false answer.

16 Discriminating Evidence Acquisition

16.1 New acquisition cohort and frozen protocol

This study tests whether selecting checks to distinguish competing evidence interpretations improves artifact reporting. The cohort is disjoint from the previously inspected 45 test capsules and 20 cached train capsules. We consider all 25 remaining capsules in the official public training manifest, using an outcome-blind acquisition plan. One archive exceeds the fixed 512-MiB download cap; the other 24 yield 41 questions. All acquired questions remain in the denominator. These are newly acquired evaluation inputs, not the benchmark's hidden test set.

The tools read published outputs, producing code, metadata, and reproduction instructions. Archives are copied into an isolated release; their code is never executed. Parsing preserves table location and notebook cells. Files larger than 10 MiB, unsupported binaries, and image-only content are not interpreted; parsed text is capped at 400,000 characters per file. The readable collection contains 3,005 documents. Every capsule has some readable output, which does not imply that every requested answer is recoverable.

Two previously inspected capsules supply the operational smoke test. The runner and formal input hashes are frozen before formal model calls; analysis choices are recorded during execution before inspecting outcomes. No policy changes follow formal results. All methods use Qwen3-8B, temperature zero, seed 20260930, and a 2,048-token output limit. Initial retrieval supplies two output passages selected from the question alone. One shared model response proposes an answer, competing interpretations, and possible inspections. Reference answers are never included in these requests.

16.2 Shared state and competing selectors

The common model prompt maintains up to three interpretations h_i with nonnegative planning weights w_i and proposes up to six next probes. For each probe a, it predicts a discrete observable outcome $z_i(a)$ under each interpretation, or marks the prediction unknown. These are heuristic forecasts, not calibrated probabilities or observations. The tested selector uses

$$u(a) = \sum_{(i,j)\in\mathcal{P}(a)} \bar{w}_i \bar{w}_j, \qquad\qquad a^* = \arg\max_a \frac{u(a)}{c(a)},$$

where $\mathcal{P}(a)$ contains pairs $i < j$ with known, different predicted outcomes. Here $\bar{w}_i$ are normalized positive weights and $c(a)$ is the declared tool cost. Ties use the shared model recommendation, then lower cost. An empty or non-discriminating forecast does not establish information gain.

The stateful validator receives the same hypotheses, probe proposals, files, accumulated observations, previous answer, and action history. It chooses the common model's recommended affordable probe. Other policies maximize $u(a)$ without dividing by cost, choose the cheapest probe, or choose a seeded random probe. All use the same subsequent model call to update the answer and state. Thus the main contrast changes selection within a model-proposed probe set; it is not a comparison to unrestricted agents or frontier models. The candidate set may itself omit the right explanation.

Output, code, and metadata searches each cost one unit and return at most 3,000 characters; expanding an observed passage costs two units and returns at most 6,000. Each policy has four units and continues until the budget is exhausted or no valid affordable unexecuted probe remains. This fixed-budget audit isolates acquisition choices; it does not validate optimal stopping. The all-initial-probes diagnostic executes the entire initial probe bank and then revises once, so it is not a matched four-unit comparator.

Table 29: New acquisition cohort, all 41 questions. Correct and wrong count closed answers; costs sum declared tool units over the cohort. The all-initial-probes diagnostic has a different budget and calls the answer model once after all probes.

Selector	Correct	Wrong	Cost
Initial report	13	9	0
Stateful validator	14	11	106
Disagreement / cost	14	11	109
Disagreement only	14	11	108
Cheapest probe	12	10	116
Random probe	14	10	111
All initial probes	14	10	124

16.3 Scoring and implementation checks

The common gate requires a non-null answer, a model completion decision, and an actual quoted output passage. Code alone cannot certify an observed result. The gate does not independently establish that a quotation supports the requested semantic scope. A separate scorer compares answers with the benchmark references using the preexisting numeric tolerance and normalized string matching. Scientific semantic fidelity remains unlabelled.

Successful closure and false termination divide correct and incorrect closed answers by all 41 questions. Recovery counts initially non-null reference-wrong answers that become correct closures. Damage counts initially reference-correct answers that become incorrect closures; loss of a correct closure also includes abstention. Shared initial calls are charged to each policy for per-policy cost but counted once in total actual requests. Tokens, returned characters, actions, and tool units are reported separately. Prefix observations are not independently optimized experiments at smaller budgets.

The primary contrast is disagreement-per-cost versus the stateful validator at the audit endpoint. We resample capsules 10,000 times and retain all questions and methods within each sampled capsule. Secondary comparisons are exploratory. No lower false-termination count alone establishes an improvement, and a zero-error bootstrap interval is not a risk guarantee. Independent implementation checks replay recorded text spans, source hashes, literal gates, budgets, input allowlists, and unique response IDs. All parse and transport failures remain in the evaluation.

16.4 Frozen pilot outcomes and mechanism diagnosis

The frozen study produces 623 actual requests with 2,867,307 tokens, no transport failures, and no malformed JSON responses. The separate smoke test uses 35 requests. Initial answers include 17 reference-correct answers, 11 non-null reference-wrong answers, and 13 null answers. Thirteen correct and nine incorrect answers initially pass the closure gate. Table 29 preserves every policy outcome.

Disagreement-per-cost and the stateful validator match all 41 final correct/wrong-closure outcomes: each yields 14 correct and 11 incorrect closures. This sample tie is not population equivalence. Neither policy, nor any other evaluated comparator, corrects an initial reference-wrong answer. No policy changes an initially correct answer into an incorrect closure; the cheapest selector loses one initially correct closure through abstention. Additional checking can therefore increase the number of incorrect closures without repairing the underlying answer.

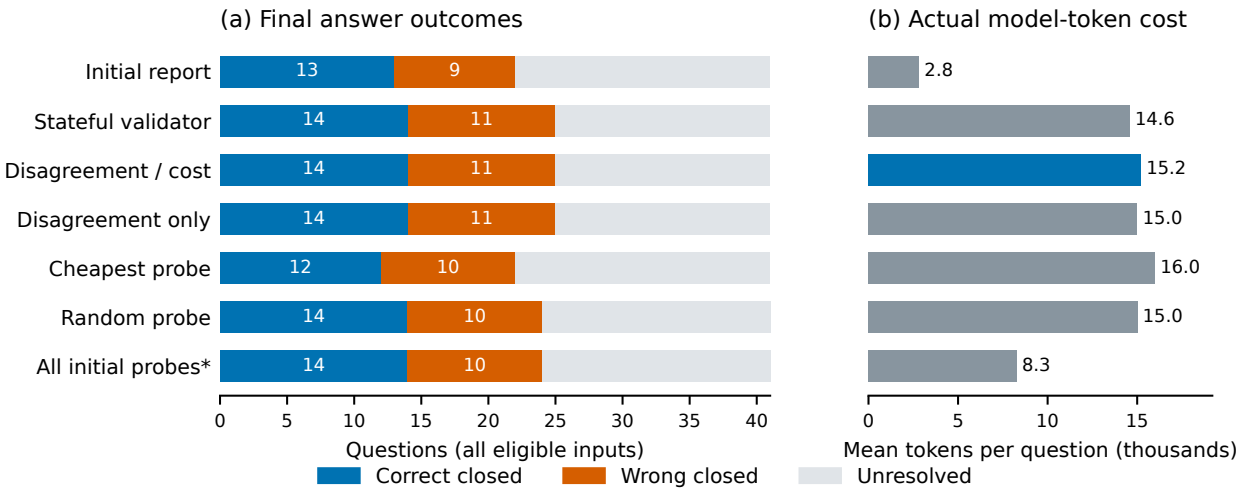


Figure 5: Frozen acquisition pilot. More inspection does not establish better answer correction. Token costs include the common initial response in each policy's allocation; the actual total counts that response once. The starred all-initial-probes diagnostic executes the initial probe bank with its own budget and only one final answer update.

The proposed selector consumes 624,028 allocated tokens versus 597,739 for the validator: a 4.4% increase. The mean paired difference is 641 tokens per question, with capsule-bootstrap interval $[13, 1449]$. The mean tool-unit difference is 0.073, with interval $[-0.024, 0.189]$. These are recorded resource costs for this execution, not isolated causal effects of the selector.

All 41 first probes match between those two policies, as do 33 complete action sequences. The model supplies only one hypothesis for 27 questions; 79 of the selector's 109 steps consequently or otherwise have zero predicted discrimination. Moreover, 59 of the validator's 106 recommendations repeat an executed query and invoke the specified first-valid-probe fallback. This limits the original comparator. Among 118 groups of byte-identical formal model requests, 45 have differing response text despite fixed seed and temperature zero. The original runs are not deterministically identical inference replicas. A separate interface diagnostic below addresses the repeated-query fallback without replacing these frozen results.

16.5 Post hoc filtered-choice diagnostic

To remove the repeated-query fallback, an extra model call chooses an exact ID from the already filtered affordable, unexecuted probe list. Both policies receive this call, and its recommendation supplies the common tie break. Hypothesis generation, scoring, tools, evidence updates, and the final gate remain unchanged. Invalid choices leave the answer unresolved. This is a post hoc diagnostic on the same 41 cases, not another untouched evaluation.

The diagnostic makes 217 additional actual requests, consuming 857,480 tokens with no transport errors. Byte-identical within-case requests are reused across policies: 416 logical post-initial calls include 199 reuses. They are not independent model runs. Three policy trajectories encounter an invalid choice, which remains an unresolved outcome rather than triggering another fallback.

The validator closes 13 correct and 11 incorrect answers. The proposed selector closes 13 and 10. Neither corrects an initial wrong answer, and both lose one initially correct closure. The one outcome difference is abstention on the same incorrect proposed model name, not answer

recovery. The paired false-termination difference is -2.4 percentage points with interval $[-7.9, 0.0]$; correct-closure difference is zero on every case. Allocated token totals are 919,775 and 919,655, respectively. The mean token difference is -2.9, with interval $[-371.9, 447.9]$. This repair does not establish a useful correction or cost advantage. Reliable candidate interpretations and outcome forecasts remain prerequisites for this selector.

17 CBR Implementation and Replay Protocol

The extension was implemented in an isolated copy of the actual 30-stage research pipeline, preserving its original working tree and source hashes. Stages 24–28 perform execution-data, citation-grounding, logic, cross-modal, and computation-trace audits; Stage 29 aggregates reports and Stage 30 applies the trust gate. The original research-decision stage already writes structured PROCEED/REFINE/PIVOT outputs. Its runner records REFINE/PIVOT history and versions stages back to their configured restart point. These features do not provide historical input-bound decision replay.

The added receipt store records policy-code identity, frozen retry context, input digests, old action, predecessor and registered local outputs. Immutable receipt files are content addressed; the active history is atomically updated. A rollback first persists its intent and invalidated suffix, then moves registered outputs into a unique quarantine directory and writes the saved checkpoint. The next invocation completes a pending rollback before replay. This recovery mechanism is tested with a prepared rollback record. It does not cancel external jobs or restore unregistered state. Inputs must remain quiescent during validation, and the store uses a single-writer lock.

17.1 Integration boundary

The opt-in Stage-15 adapter captures the existing requirements-gate routing decision after the judge returns. Its canonical verdicts are PROCEED, PARTIAL, and REJECT; a remaining retry converts REJECT to REFINE. The original rule can select PROCEED after retry exhaustion despite unmet requirements. We preserve this behavior: replay correctness is not policy safety. Updating a registered verdict witness triggers counterfactual replay before the runner resumes. Upstream evidence-to-judge recomputation remains the producer's responsibility. General free-form LLM decisions lack a frozen, deterministic replay adapter here and are outside the evaluated guarantee.

Ten focused behavior tests cover multiple flips, PIVOT parsing through the original parser, earliest rollback, unchanged-action refresh, unrelated changes, missing witnesses, exhausted replay budgets, policy changes, receipt tampering, path containment, persistence, and pending rollback recovery. A separate integration test executes the real Stage-15 function with four controlled judge/retry combinations, verifying both its structured output and captured receipt. These mocked-judge tests are software tests, not additional model calls. An initial adapter-enumeration mismatch was fixed after inspecting the real judge; the initial run is retained and the unchanged experiment was rerun under freshly frozen code hashes.

17.2 Executed branch trace

In an archived two-decision scenario, both decisions depend on a changed source. Their old actions are (REFINE, PROCEED). Replay leaves the first action unchanged but changes the second to REFINE. CBR retains the first output, quarantines the second, and persists a checkpoint identifying the second decision. Full replay produces the same result. The existing tracker correctly marks both dependent certificate checks false, but its interface does not revise the saved PROCEED action. This is a controlled version change on a real artifact, not an observed natural research-policy error.

17.3 Accounting and limitations

There are 45 capsules, 405 event scenarios and 711 historical decisions per policy. Source changes and restorations produce 326 changed actions in 254 scenarios. Four policies yield 1,620 policy-

Table 30: Fixed-retrieval SciFact operating points (%). Valid-claim recall means correct SUPPORTS/REFUTES resolution. Selective risk counts every wrong closed relation. Across the five saved runs, each LLM-containing policy has identical labels, so all operating-point sample standard deviations are 0.0; confidence rankings vary. Zero run variance does not imply zero sampling uncertainty.

Policy	Coverage ↑	NEI-FCR ↓	Valid-claim recall ↑	Macro-F1 ↑	Selective risk ↓
Direct NLI	58.1	33.8	56.0	59.5	40.6
LLM judge	72.6	46.9	64.6	60.2	45.1
Rule auditor	34.5	14.6	38.8	54.6	30.8
Risk-controlled NLI	27.7	9.2	34.0	53.2	24.5
Mondrian conformal (90%)	11.2	3.1	16.3	39.8	10.5
NLI–LLM consensus	30.4	10.8	36.8	54.9	25.2
Typed NLI only	41.0	12.3	52.2	65.3	21.6
TrustScientist	34.8	8.5	46.9	63.5	16.9

event records. Independent code reconstructs file bytes and verifies action labels, receipt digests, checkpoint targets, output presence after rollback, and summary arithmetic. All cases are retained. Existing tracker work is 1,530 predicate evaluations; CBR's 382 and full replay's 416 are decision evaluations and should not be treated as interchangeable cost units. The 8.2% reduction is an observed work-count ratio, not a latency or population-effect claim. The experiment does not measure model tokens because no model is called.

The linear histories and version interventions are authored. Every event starts from the prior file state with newly recorded receipts; the experiment does not resume an autonomous research trajectory after each rollback. The registered expected versions are a contract oracle, not evidence that the underlying scientific claims are true. Complete read sets and frozen policies are assumptions. An augmented dependency tracker with action replay and the same rollback primitive can reproduce CBR; the current study does not demonstrate a new general incremental-computation algorithm.

17.4 Assistance and artifact

The author's October 2 request specified auditing the seven added stages, implementing affected-decision replay with earliest-flip rollback, comparing four named policies, and updating the algorithm and artifact. Codex assisted with implementation, controlled experimental design, scoring and writing. The request, frozen experiment protocol, model-free test scope, source hashes, and reproduction commands are included in the artifact. No generated labels are described as independent human validation.

18 Preserved Protocol Details

18.1 Selective revalidation of changed inputs

We additionally implement a dependency tracker for versioned certificates. Let I_t be the checks directly bound to changed files. The invalidated set is

$$U_t = I_t \cup \mathrm{Desc}_G(I_t). \tag{5}$$

The tracker clears cached decisions in U_t, re-evaluates them in topological order, and retains the other checks. For deterministic predicates with complete declared dependencies, this yields the same decisions as full re-audit: inputs outside U_t are unchanged, and induction over the affected order recomputes every changed dependency. Hidden inputs or changing verifiers invalidate this argument. This is standard incremental dependency tracking [2]; our experiment tests its certificate-lifetime role. The tracker is evaluated separately from the original acquisition loop, which continues to re-audit its full small graph.

19 SciFact: A Fixed-Retrieval Semantic Slice

19.1 Task, controls, and metrics

SciFact annotates whether scientific abstracts support, refute, or provide insufficient evidence for a claim [4]. The evaluator uses 339 unique development claim–document pairs: 138 SUPPORTS, 71 REFUTES, and 130 NEI (not enough information). Thresholds are chosen on 919 training pairs and frozen. In this setting, CLOSED means that the support/refutation relation is resolved. A correct REFUTES decision counts as correct closure while rejecting the claim's asserted relation to the evidence. It does not grant publication acceptance.

Every policy receives the same top-ranked sentence from one frozen DeBERTa-v3 natural language inference (NLI) model. NLI scores entailment, contradiction, and neutrality between a sentence and a claim. NLI-only policies use one scoring pass; LLM policies can add one temperature-zero `deepseek-chat` call. The appendix records the exact NLI checkpoint. No policy obtains additional evidence. The selected sentence matches a gold rationale for only 56.5% of evidence-bearing pairs. Gold labels refer to the source evidence, so end-task errors can involve retrieval as well as relation prediction. The fixed evidence makes policy comparisons comparable but does not isolate these two causes causally.

Coverage is the fraction of all 339 pairs assigned SUPPORTS or REFUTES. NEI false-closure rate (NEI-FCR) is the fraction of the 130 gold-NEI pairs closed. *Valid-claim recall*, retained as a metric name for compatibility with the original evaluation, means correctly resolved gold SUPPORTS/REFUTES pairs divided by 209; it does not mean recall of true scientific propositions. Selective risk is the fraction of closed pairs whose relation is wrong, including both NEI closures and SUPPORTS/REFUTES reversals. Macro-F1 averages three-way class F1, mapping unresolved predictions to NEI. Macro-F1 is not a selective safety metric. These definitions make 98 correct closures among 118 total selections correspond to 46.9% valid-claim recall and 16.9% selective risk.

The policies include direct NLI, an LLM judge, a high-confidence rule auditor, a risk-controlled NLI rule, and 90% class-conditional Mondrian conformal prediction [5, 6]. Global NLI–LLM consensus uses a common relation threshold; typed NLI uses separate support and refutation thresholds, 0.35 and 0.975. TrustScientist adds agreement with the LLM on the selected relation. Source admissibility is fixed, so this slice tests relation-specific selection and agreement. It does not separately exercise the four artifact-obligation types.

19.2 Utility, errors, and the consensus comparator

TrustScientist closes 118/339 pairs, including 98 correctly resolved evidence-bearing pairs. Its 20 wrong closures comprise 11 NEI cases and nine SUPPORTS/REFUTES reversals. Reporting only NEI-FCR would hide nearly half of these selected errors. Typed NLI attains higher Macro-F1 and recall; adding agreement lowers NEI closures from 16 to 11 while reducing correct relation closures from 109 to 98. This is an error–utility tradeoff, not dominance.

Global consensus correctly closes 77 evidence-bearing pairs and closes 14 NEI pairs. TrustScientist retains more correct decisions, but the difference of three NEI cases does not establish a safety advantage over consensus. Mondrian conformal prediction has the lowest NEI-FCR, four cases, while correctly closing only 34 evidence-bearing pairs. The supplied conformal configuration is a comparator; we make no transfer of its calibration guarantees to TrustScientist or post-action data.

The original matched-coverage analysis spans only 5–27.5% coverage, below TrustScientist's 34.8% operating point. It integrates NEI-FCR, excluding relation reversals, and normalizes by the interval length. Its reported values, 0.0241 for TrustScientist and 0.0260 for global consensus, are numerically close. Clipped ranking scores also create ties whose order follows the saved records. The main paper retains the original matched-coverage curves; the appendix records their protocol. They do not establish a general risk–coverage advantage.

A post hoc rationale-hit diagnostic further separates the observed errors. On 118 evidence-bearing pairs whose sentence matches a gold rationale, TrustScientist makes 78 correct closures and one wrong relation; on 91 rationale misses it makes 20 correct closures and eight wrong relations. This stratification locates an association with evidence selection. It is not an oracle-retrieval experiment and does not assign errors causally.

All 11 NEI false closures involve agreement between typed NLI and the LLM on the same non-NEI relation. Ten also pass global consensus with NLI scores above 0.98. One retrieved fragment contains a section cross-reference rather than a proposition and passes the lower support threshold. The examples show two limits: model agreement can preserve shared semantic mistakes, and a fixed retriever can supply evidence that cannot resolve the claim. An expert audit of those errors is distinct from their external SciFact labels; this study does not report a new human-validation experiment.

20 Recorded Result Visualizations

Figures 6–9 display the recorded cohorts with their original denominators. They add no new experiment. The figures distinguish branch consistency, certificate validity, benchmark-answer agreement and semantic relation resolution. These endpoints are not interchangeable measures of scientific correctness.

20.1 Reproduction and interpretation

The code and recorded evidence are available in the anonymous repository. The artifact contains the portable CBR implementation, its archived inputs and outputs, selected recorded cases, and the frozen inputs and plotting scripts for these figures. The CBR experiment runs offline on Linux with the Python standard library. The repository delivery uses split archives, unpacked by the first command below. The result figures require Matplotlib and NumPy; their data and scope captions are saved alongside the scripts. Model outputs are retained records, not independent human labels. The source manifest distinguishes the full offline CBR replay from recomputing statistics over previously recorded model outputs. New model inference and unrestricted autonomous scientific repair are not reproduced by the offline commands.

```
python unpack_artifact.py
python test_cbr.py
python run_branch_experiment.py prepare
python run_branch_experiment.py
python validate_branch_results.py
```

20.2 Maintenance and failure modes

CBR and full replay find all recorded earliest rollback points; the certificate-only tracker has no replay or rollback operation. The graph and flat dependency representations have identical certificate outcomes and predicate counts. Reduced work counts do not establish lower total latency, model cost or scientific verification cost. The acquisition panels retain the structured-selector tie and the semantic scope gate's loss of correct closures. Risk-qualified acquisition closes no case after the failed historical calibration.

At budget four, risk-qualified closures are 0/82. The same fresh-workspace tasks succeed in 19/20 cases, with 58/59 fresh answer matches in each saved run. Prepared compatibility fixes and the repeated task set limit the interpretation of this diagnostic.

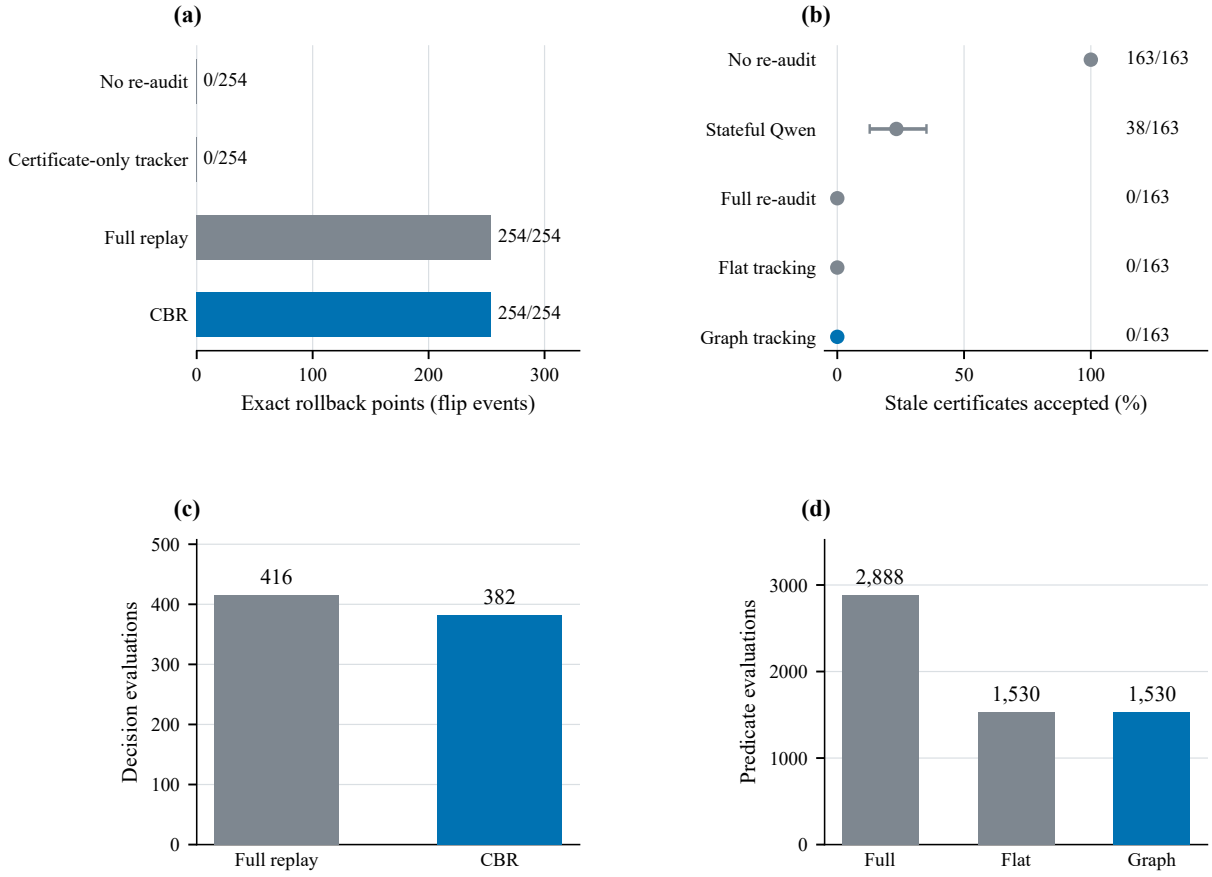


Figure 6: Controlled maintenance studies: 45 capsules and 405 update scenarios. CBR means Counterfactual Branch Revalidation. (a) Exact earliest rollback among 254 flip scenarios; the certificate-only tracker is the archived implementation without replay/rollback. (b) Stale certificate acceptance among 163 opportunities; whiskers are existing 45-capsule bootstrap 95% intervals. Degenerate empirical zero intervals are not population guarantees. Full, flat and graph retain 332/332 valid certificates; Qwen3-8B retains 327/332. (c) Decision evaluations for the two correct branch-replay policies. (d) Certificate predicate evaluations for equivalent deterministic policies. These separate work units exclude initialization and are not end-to-end costs. Histories are authored, changes scripted and dependencies supplied.

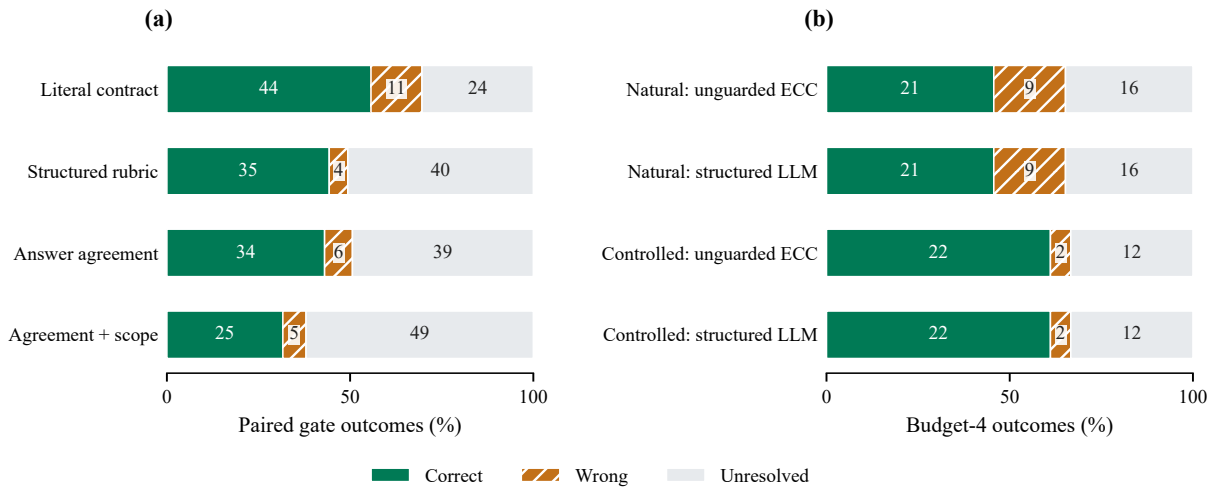


Figure 7: (a) Outcomes for 79 paired semantic-gate questions; correct/wrong mean benchmark-answer agreement, not expert scientific validation. Correct/Wrong count closed cases; Unresolved means not closed. (b) Budget-four existing-artifact acquisition on 46 natural reporting questions and 36 controlled states. ECC (Evidence-Closure Control) is unguarded here. Budget units are declared tool costs, not time or currency: rebinding/extraction costs one, search/fetch two, and broad refresh three. The structured large-language-model selector matches all 82 paired outcomes. The guarded controller closes no cases after failed calibration. Counts sum to each cohort denominator. Panels are not pooled; unrestricted autonomous recovery is untested.

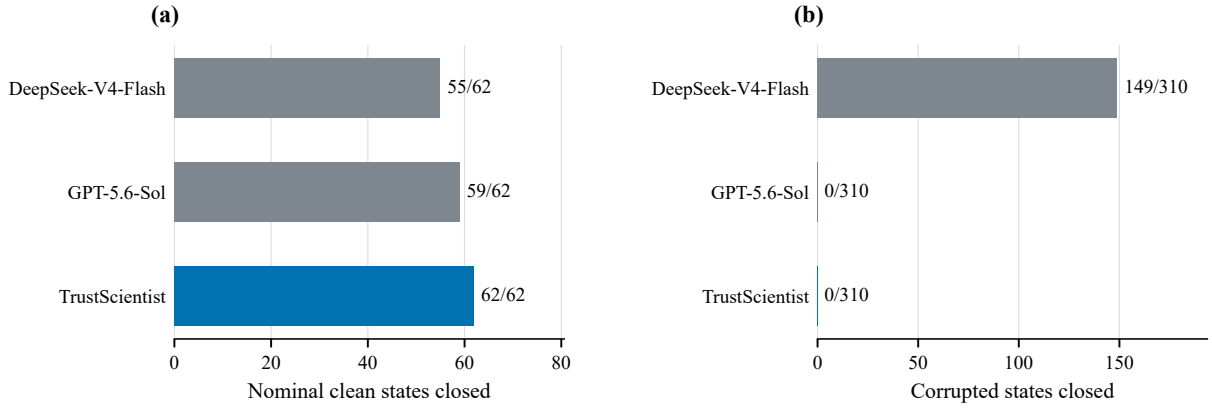


Figure 8: Matched supplied manifests: 62 nominal clean and 310 controlled corrupted states per policy. Clean closure is not semantic binding accuracy: three GPT nonclosures expose mismatches that TrustScientist accepts. Corrupted closure uses all 310 states, not a clean-closed conditional denominator. Models receive supplied validity flags, not raw artifacts. These are functional stress tests, not a natural-error or SOTA estimate.

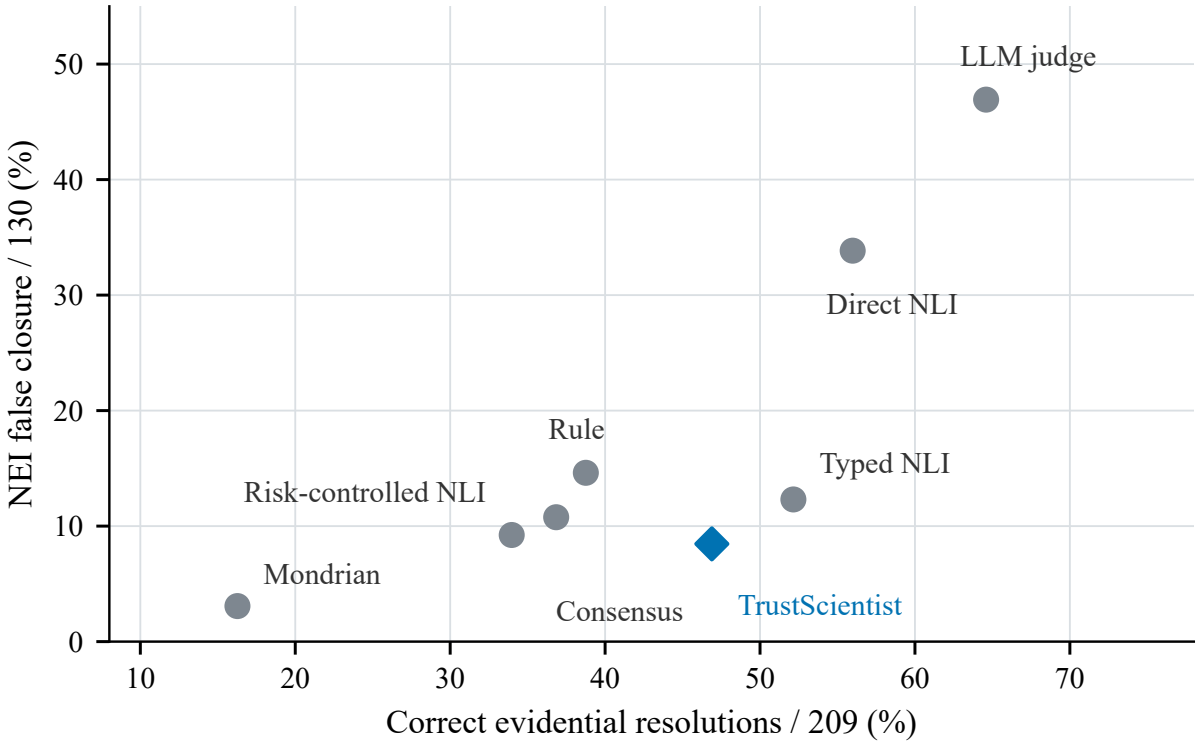


Figure 9: Fixed-retrieval SciFact: 339 pairs, with 209 evidence-bearing pairs and 130 NEI (Not Enough Information) controls. CLOSED resolves SUPPORTS/REFUTES, not publication acceptance. Mondrian uses class-conditional conformal prediction at alpha=0.10 (90% nominal coverage); risk-controlled NLI uses a calibration risk target of 0.20. The six repeated policies use a representative saved run; their five-run aggregate operating-point metrics are identical. The two additional policies use single saved deterministic outputs. Methods have different closure coverage: points are not a matched-coverage frontier or connected curve. Lower NEI false closure can reduce useful resolutions.

Table 31: Totals across 216 controlled updates from 36 histories, excluding common initialization. Verification includes the policy call made while issuing each new receipt; it is charged equally. Fits count actual continuation training calls in the primary run.

Policy	Re-audit	Verify	Continue	Fits
Full replay	2,110	904	1,320	135
Generic replay	485	904	1,320	135
TrustScientist-CBR	581	904	1,320	135

21 Executed Longer Histories and Continuation

The new study uses a scripted analysis policy, actual classifier fitting, controlled evidence changes, and executed continuation after rollback. It extends the original CORE artifact replay test without changing its population or results. No new foundation-model call or human annotation is used. The four public dataset families have appeared in earlier experiments; new split seeds do not constitute new datasets.

21.1 Histories, updates, and independent scoring

There are 36 histories: four datasets, three stratified split seeds and three search lengths. One, five or eleven distinct regularization settings produce 4, 12 or 24 decisions, respectively. Each fitted candidate contributes an admissibility check and a baseline comparison. Dataset availability and a terminal report gate complete the history. The terminal gate requires an admissible candidate with macro-F1 at least 0.90 and no worse than the current baseline. These thresholds define the frozen routing rule; they are not a general standard of scientific adequacy.

All six updates are retained. Early/late revisions and the baseline revision use newly executed fits. Training-score substitution and deletion are controlled fault interventions. Unchanged content measures branch retention. Each episode begins from the same prepared history for every method; the episodes do not form a naturally evolving research project.

After an action flip, registered decision outputs and publication products are quarantined. The suffix resumes from the saved index. An inadmissible candidate is refitted on the held-out split; a failed comparison permits one revised-configuration fit. New prediction arrays, decisions, CSV, SVG and Markdown products are written. Re-audit after continuation refreshes remaining changed attestations. Model fits in continuation are forced for every replay method, even when an identical prediction bundle already exists.

The scorer uses its own macro-F1 calculation, policy interpretation and product parsers. It checks the earliest rollback, the saved checkpoint and quarantined files. It compares final actions, CSV values, chart bar widths and the report's selected candidate with saved predictions. Final consistency includes unchanged episodes and is distinct from executing a rollback in every episode.

21.2 Accounting and matched baseline

CBR's total work is 2,805 policy entries versus 4,334 for full replay. The 35.3% difference includes verification and continuation, beyond the replay-only count. It remains a work metric, not dollars or unrestricted end-to-end cost. Balanced timing runs each of the three replay methods twice on the same 216 episodes, distributing every method equally across first, second and third positions. Post-update time includes refitting, version checks, receipt persistence and publication writes; common initialization is excluded. The two repetitions total 12.38 seconds for full replay, 11.04 for generic

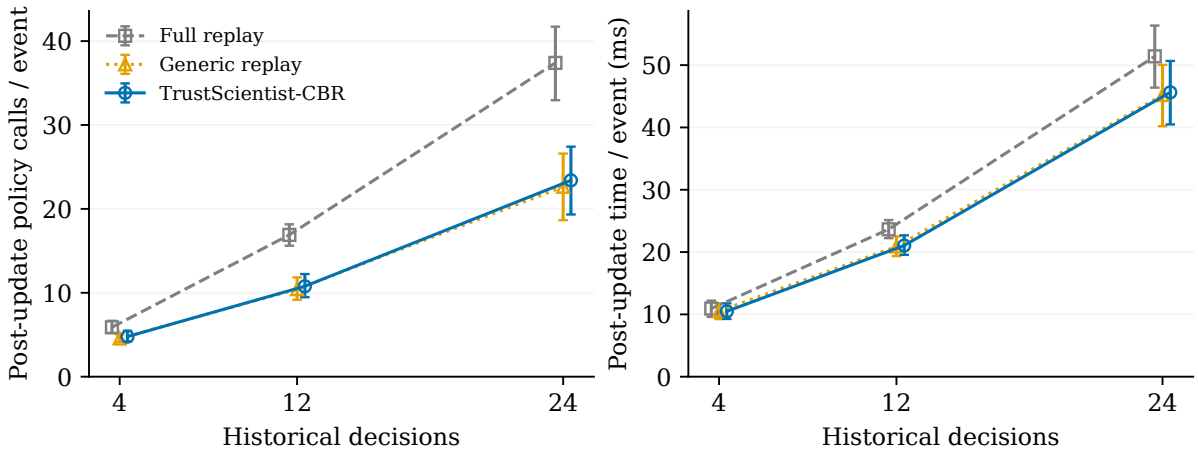


Figure 10: Work and measured time on executed scripted histories. Each length contains 72 paired events (controlled updates) on 12 dataset–split clusters. Policy calls include re-audit, receipt verification and continuation. Horizontal offsets separate overlapping methods. Only the three methods with consistent final outputs on all episodes are shown. Points are means; whiskers are 95% cluster-bootstrap intervals. Time uses two balanced repetitions of the same cases. Common initialization is excluded; continuation includes actual model refits. Generic replay matches correctness and uses fewer policy calls than CBR.

replay, and 11.11 for CBR. The local CPU setting and small policies limit extrapolation to expensive LLM decisions or external jobs.

The initial completed version gave the generic planner an additional file read before the shared engine verified versions. That run is preserved. The final version supplies every planner the same verified versions and reruns continuation and timing, leaving data, thresholds and policies fixed. The generic planner independently scans per-receipt attestations and shares the same rollback/continuation machinery. It matches correctness with fewer policy entries than CBR. This is a local generic baseline, not ClaimGarden code or a reproduction of its complete workflow.

21.3 An executed rollback and continuation

The Digits trace has 24 decisions. A late candidate's held-out result is replaced by its actual training-score artifact. Its previous admissibility decision was PROCEED; replay changes it to REFINE because the split is now `train`. CBR preserves the 21 earlier decisions, quarantines the suffix and its publication products, and restores the checkpoint before the failed admissibility decision. Full replay and generic replay select the same index.

The continuation performs a new held-out fit, writes its prediction arrays and artifact, reruns the suffix checks, and issues new decision receipts. The table, chart and report are regenerated from current predictions. The substituted training macro-F1 is 1.000; the newly fitted held-out score is 0.9550. CBR and generic replay each make eight post-update policy calls; full replay makes 29, including receipt verification and continuation. The independent scorer verifies these products and the absence of a stale final action. The archive's zero-based rollback index is 21; the source record and hash are in `revision_trajectory/worked_trace.json`. This trace executes the sequence evidence update, stale action, selective replay, rollback and corrected publication, under a deterministic scripted policy. It does not establish a naturally observed scientific failure.

21.4 Original agent-history gap

Five archived AutoResearchClaw runs contain ten recorded REFINE/PIVOT history entries and 99 stage-decision files, but no complete version-bound decision receipt. The runner did not log every PROCEED in the history, and archived decisions lack frozen input versions and policy context. Those materials cannot establish full natural-history replay correctness. The present study tests executed analysis operations with constructed histories and updates. Natural error prevalence, unrestricted model redecision, and broad scientific validity remain unmeasured.

References

[1] Umut A. Acar, Matthias Blume, and Jacob Donham. A consistent semantics of self-adjusting computation. *Journal of Functional Programming*, 23(3):249–292, 2013. doi: 10.1017/S0956796813000099.

[2] Andrey Mokhov, Neil Mitchell, and Simon Peyton Jones. Build systems à la carte: Theory and practice. *Journal of Functional Programming*, 30:e11, 2020. doi: 10.1017/S0956796820000088.

[3] Anastasios N. Angelopoulos, Stephen Bates, Emmanuel J. Candès, Michael I. Jordan, and Lihua Lei. Learn then test: Calibrating predictive algorithms to achieve risk control, 2021. URL `https://arxiv.org/abs/2110.01052`. Version 5, revised 29 September 2022.

[4] David Wadden, Shanchuan Lin, Kyle Lo, Lucy Lu Wang, Madeleine van Zuylen, Arman Cohan, and Hannaneh Hajishirzi. Fact or fiction: Verifying scientific claims. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 7534–7550, Online, 2020. Association for Computational Linguistics. doi: 10.18653/v1/2020.emnlp-main.609. URL `https://aclanthology.org/2020.emnlp-main.609/`.

[5] Stephen Bates, Anastasios Angelopoulos, Lihua Lei, Jitendra Malik, and Michael Jordan. Distribution-free, risk-controlling prediction sets. *Journal of the ACM*, 68(6):1–34, 2021. doi: 10.1145/3478535. URL `https://doi.org/10.1145/3478535`.

[6] Anastasios N. Angelopoulos and Stephen Bates. Conformal prediction: A gentle introduction. *Foundations and Trends in Machine Learning*, 16(4):494–591, 2023. doi: 10.1561/2200000101. URL `https://doi.org/10.1561/2200000101`.